Artificial Intelligence Review (2026) 59:129
https://doi.org/10.1007/s10462-026-11532-7

Graph neural networks for anomaly detection: a systematic review of dynamic temporal approaches

Fernando Ares-Robledo[1] · **Helena Rifà-Pous**[1] · **Robert Clarisó**[1]

Received: 30 July 2025 / Accepted: 19 February 2026 / Published online: 11 March 2026

Abstract

Graph Neural Network (GNN) have recently gained significant attention for their ability to model evolving relationships in graph-structured data, offering new opportunities for anomaly detection in cybersecurity. This Systematic Literature Review (SLR) examines the current state of research on these models applied to cybersecurity-related anomaly detection tasks. We systematically analyze 79 studies to identify key trends, challenges, and opportunities in this emerging field. Our review highlights that while GNN offer unique advantages such as capturing spatiotemporal dependencies and modeling complex cyber-threat patterns, several barriers remain, including scalability issues, limited real-world datasets, and the lack of interpretability in most models. Common architectures such as Graph Convolutional Networks (GCN), Graph Attention Networks (GAT), and hybrid Transformer-based models are identified, along with their applications in intrusion detection, botnet detection, and anomaly detection in industrial control systems. Several studies propose that GNN can overcome current limitations by integrating contrastive learning and adaptive models for real-time threat detection. This review emphasizes the need for scalable, explainable, and deployment-ready solutions to fully realize the potential of dynamic graph models in cybersecurity. Future research should focus on developing scalable and adaptive architectures, integrating improved interpretability mechanisms, and enhancing model robustness through cross-domain validation and real-time applications.

Keywords Cybersecurity · Anomaly detection · Spatiotemporal graphs · Graph neural networks

✉ Fernando Ares-Robledo
faresro@uoc.edu

Helena Rifà-Pous
hrifa@uoc.edu

Robert Clarisó
rclariso@uoc.edu

[1] Internet Interdisciplinary Institute (IN3), Universitat Oberta de Catalunya (UOC), Rambla del Poblenou, 154, 08018 Barcelona, Catalunya, Spain

1 Introduction

One of the critical challenges in network cybersecurity is intrusion detection, especially due to the continuous evolution and emergence of novel threats over time (Khraisat et al. 2019). A fundamental approach to counter these threats is anomaly detection, which typically relies on Artificial Intelligence (AI) systems designed to learn patterns of normal network traffic and identify deviations as potential anomalies, indicative of intrusions (Mishra et al. 2019). Although extensive research has been conducted on AI-based anomaly detection over the past decades, deploying these solutions effectively in real-world scenarios remains challenging due to several critical limitations. Current anomaly detection models often suffer from significant overfitting, performing well on specific datasets but failing to generalize to diverse or unseen network environments (Diukarev and Starukhin 2024). Additionally, these models typically require large-scale training datasets, which may be impractical or unavailable in many real-world scenarios (Pinto et al. 2023). Another considerable limitation is the difficulty of adapting existing models to handle concept drift, where the statistical properties of the network traffic change over time, thus rendering previously learned patterns obsolete (Gama et al. 2014).

In recent years, new approaches have emerged that aim to address these challenges by exploring alternative modeling perspectives. One promising direction involves Graph Neural Network (GNN), which have gained attention due to their inherent capability to represent relational structures within network data. These models offer flexibility in capturing complex relationships, adapting well to different network scenarios, and reducing the reliance on extensive training datasets. Their relational data representation provides an effective framework for detecting anomalies that evolve with network traffic dynamics. Representative tasks addressed by these models include classification, community detection, link prediction, graph embedding, anomaly detection, and imputation, as illustrated in Fig. 1. The versatility of these approaches extends far beyond cybersecurity, finding applications in numerous domains.

1.1 Related work

The increasing interest in analyzing graph-structured data through advanced machine learning methods has resulted in several systematic reviews exploring different facets GNN and related methodologies. General surveys, such as those by Khemani et al. (2024) and Wu et al. (2021), provide comprehensive introductions to the fundamental concepts, architectures, and a broad spectrum of application scenarios, establishing baseline knowledge and serving as foundational references in the field.

Surveys focusing explicitly on dynamic graphs have emerged in recent years. For instance, the surveys covering thoroughly dynamic GNN models (Pang et al. 2022; Zheng et al. 2024), proposing classifications based on temporal information integration, while other studies evaluate training frameworks and benchmarks various dynamic graph architectures (Feng et al. 2024). Meanwhile, the investigation by Barros et al. (2021) specifically targets graph embeddings, analyzing the impact of temporal granularity, topological evolution, and embedding methods, and provides taxonomies categorizing techniques from matrix decomposition to deep learning approaches.

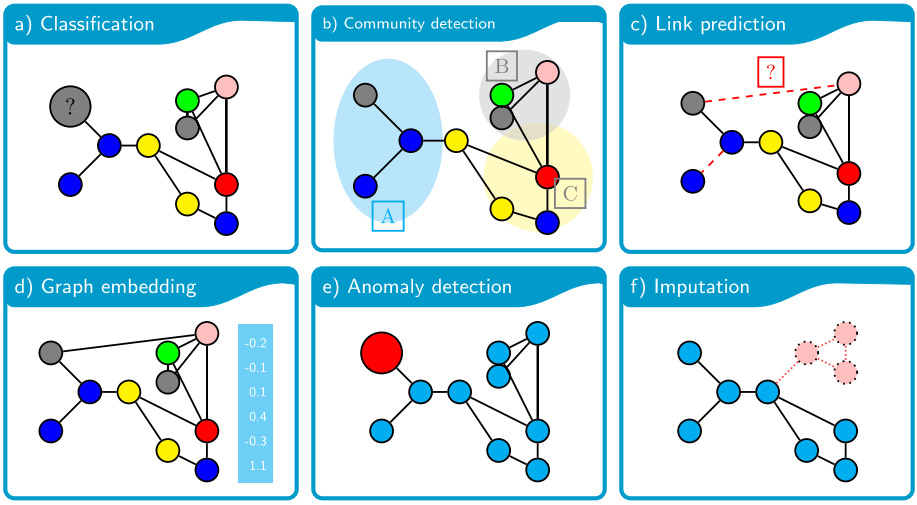

Fig. 1 Representative GNN tasks in dynamic graphs: **a** classification, where nodes are labeled by color for different categories; **b** community detection, highlighting overlapping subgroups; **c** link prediction, indicating potential or missing edges with dashed lines; **d** graph embedding, mapping node features into a low-dimensional vector space; **e** anomaly detection, isolating suspicious nodes or edges that deviate from normal patterns; and **f** imputation, inferring unknown nodes or links in partially observed data

In domain-specific contexts, Yan et al. (2023) review graph applied mining methods, analyzing tasks like fraud identification. Similarly, Jiang and Luo (2022) provide an overview applied to various traffic forecasting problems, emphasizing their unique suitability in modeling spatial dependencies inherent to transportation networks; Shlomi et al. (2021) discuss applications of GNN within particle physics. In addition, GNNs have been harnessed for molecular property prediction, scoring and docking, optimization, and dynamics simulation (Wang et al. 2023).

Furthermore, Fard et al. Fard (2024) explore less conventional applications of graph learning, such as air quality analysis, tracking objects, and other novel tasks not traditionally associated with GNN, emphasizing their potential in diverse and emerging areas.

Although these reviews collectively offer comprehensive insights into GNN methodologies and specific domain applications, they often overlook critical aspects relevant for practical cybersecurity deployments. To our knowledge, this is the first systematic literature review to focus specifically on the application in the cybersecurity domain, thus explicitly addressing this gap. Table 1 summarizes and contrasts previous systematic reviews with our own, emphasizing our distinctive focus and analysis dimensions.

1.2 Our contributions

In this systematic literature review, we focus specifically on GNN for anomaly detection in cybersecurity. Our goal is to provide a comprehensive overview of current research trends, identify critical gaps, and suggest directions for future research. To guide our review, we address the following research questions:

RQ1: What does the publication landscape look like?

RQ2: Which learning and inference paradigms are most commonly adopted, and what

Table 1 Comparative analysis of existing surveys/SLRs on GNN

Survey / SLR	Avg. Year	Domain	GNN types	Method of comparison	Architecture details
Jiang and Luo (2022)	2019	Traffic	GCN, GAT	Experimental	✓
Barros et al. (2021)	2020	Multi-domain	For embeddings	Taxonomy-based	△
Yan et al. (2023)	2020	Malware/Fraud detection	General GNN	Taxonomy-based	△
Feng et al. (2024)	2022	General/Multidomain	General GNN	Experimental	✓
Zheng et al. (2024)	2021	General	General GNN	Taxonomy-based	✓
Fard (2024)	2021	Multidomain	General GNN	Application-based	×
Wu et al. (2021)	2019	General	GCN, GAT, GAE	Taxonomy-based	✓
Khemani et al. (2024)	2020	General	GCN, GAT, GraphSAGE	Conceptual	△
Shlomi et al. (2021)	2020	Particle Physics	General GNN	Application-based	△
Wang et al. (2023)	2021	Molecular Sciences	MPNN, SE(3)-GNN	Application-based	✓
Our SLR	2023	Cybersecurity	GNN-based Anomaly Detection	Systematic/Thematic & Taxonomy-based	✓

The symbol "✓" indicates that the survey covers or compares that aspect in detail; "△" indicates partial coverage or superficial comparison; and "×" means it is not covered

trade-offs do they imply for cybersecurity data?
RQ3: Which GNN architectural families dominate dynamic/temporal cybersecurity anomaly detection, and how is temporal information integrated or exploited over time?
RQ4: How are graphs constructed and at what analysis granularity do approaches operate, and how does granularity affect modeling choices?
RQ5: How are approaches evaluated, and what recurrent threats to validity or biases arise from these choices?
RQ6: What practical barriers remain open, and which research directions are proposed to address them?
Table 2 links each results subsection to the research question(s) it addresses.

The remainder of this paper is organized as follows. Section 2 provides an overview of the theoretical foundations. Section 3 details the methodology adopted for our systematic literature review, including search strategy, inclusion criteria, and data extraction. In Sect. 4, we analyze the research panorama, highlighting publication trends and key journals. In Sect. 5, we present the results of our review, highlighting key trends, architectural choices, and performance metrics. Section 6 discusses the main findings and their implications for future research. Finally, Sect. 7 concludes the paper by summarizing the key insights.

2 Background

This section provides a comprehensive and structured foundation for understanding GNN in the context of anomaly detection. We first introduce formal definitions critical to modeling graph-structured data, enabling precise representations of dynamic network states. The discussion then explores different perspectives for analyzing graph-structured data. Key GNN

Table 2 Mapping between manuscript sections and research questions

RQ	Main results section(s)	Discussion
RQ1	Sect. 4 (Research panorama)	Sect. 6.1
RQ2	Sect. 5.1.1 (Supervision paradigms); Sect. 5.1.2 (Training scope)	Sect. 6.2
RQ3	Sect. 5.2.1 (Architectural trends); Sect. 5.2.2 (Adaptive models over time)	Sect. 6.3
RQ4	Sect. 5.3.1 (Graph analysis levels); Sect. 5.3.2 (Rationale for granularity)	Sect. 6.4
RQ5	Sect. 5.4.1 (Validation strategies); Sect. 5.4.2 (Datasets); Sect. 5.4.3 (Metrics and potential biases)	Sect. 6.5
RQ6	Sect. 5.5.1 (Code availability); Sect. 5.5.2 (Real-world case studies vs. proofs of concept); Sect. 5.5.3 (Limitations and future directions); Sect. 5.5.4 (Domains motivating graph-based systems)	Sect. 6.6

tasks, including node classification, link prediction, and anomaly detection, are reviewed to underscore their relevance to the identification of cybersecurity threats. Finally, we examine prominent GNN architectures, such as Graph Convolutional Networks (GCN), Graph Attention Networks (GAT), and hybrid models, assessing their strengths in capturing spatial and temporal dependencies essential for real-time anomaly detection.

2.1 Formal definitions

Essential concepts of graphs in dynamic anomaly detection are defined below.

Graph: A graph can be formally defined as $G = (V, E, X)$, where V is the set of nodes (or vertices), $E \subseteq V \times V$ is the set of edges, and X is an optional feature matrix. If each node $v \in V$ is associated with a feature vector x_v, then X can be viewed as the collection $\{\mathrm{x}_v : v \in V\}$. Edges may also carry features, which can be captured in an analogous manner.

Neighbors: In a graph $G = (V, E)$, the neighbors of a node $v \in V$ are those nodes directly connected to v by an edge. This concept underlies the local message-passing mechanism (discussed in Paragraph 2.1) in GNN architectures, where information is iteratively exchanged between each node and its neighbors.

Adjacency matrix: A common way to represent the structure of a graph with n nodes is through an adjacency matrix $A \in \mathbb{R}^{n \times n}$, whose entries indicate which pairs of nodes are connected. From A one can derive the diagonal degree matrix D, whose ith entry encodes the degree of node v_i; in GNN, A and D are typically combined to build normalized propagation operators so that highly connected nodes do not dominate feature aggregation.

(TS) and Multivariate Time Series (MTS): A TS is a sequence of real-valued observations $\{x_t\}_{t=1}^{T}$ indexed by time, each $x_t \in \mathbb{R}$ describing the state of a single variable (for example, CPU load or packet count). In typical cybersecurity settings, several signals are monitored concurrently, yielding a MTS $\{\mathrm{x}_t\}_{t=1}^{T}$ where $\mathrm{x}_t \in \mathbb{R}^n$ stacks the n variables at time t.

Temporal graph: A temporal graph explicitly encodes time by attaching timestamps to edges (or nodes), or by organizing nodes into time-indexed layers. Thus, the graph records

not only which interactions occur but also when they occur, which is crucial in cybersecurity scenarios where ordering and frequency of events reveal attack patterns.

Dynamic graph: A dynamic graph evolves over time through changes in nodes, edges, or features, and can be represented as a sequence of snapshots $\{G(t)\}_{t=1}^{T}$, where each $G(t) = (V(t), E(t), X(t))$ may differ from $G(t-1)$. Such evolution is typical in network-security environments, where configuration changes, user behavior, or attacks continually modify the underlying topology, as illustrated in Fig. 2.

GNN: Graph neural networks extend deep learning to relational settings by operating directly on a graph and iteratively updating each node representation through the aggregation of information from its neighbors. After several layers, the resulting node embeddings encode both local attributes and a broader structural context that spans several successive steps of message exchange. In the field of cybersecurity, these embeddings act as dynamic risk fingerprints for hosts, flows, or alerts, which enables anomaly detection at the level of nodes, edges, or entire graphs. For a formal treatment of the original GNN model, we refer to Scarselli et al. (2009).

Message-passing: In most GNN, the message-passing process alternates between an aggregation step and an update step. In each layer, every node gathers information from its neighbors, aggregates these messages into a single summary, and then merges this summary with its current representation to obtain a refined embedding. The use of several consecutive layers allows the information to travel across neighborhoods reached after several successive steps, so that deeper layers encode increasingly extensive regions of the graph, as shown in Fig. 3.

Embeddings: After several rounds of message-passing, each node is associated with a final hidden vector that we regard as its embedding, summarizing both its intrinsic features and multi-hop structural context. Stacking these node embeddings produces a low-dimensional representation of the whole graph, which subsequent sections exploit for node-, edge-, or graph-level anomaly detection.

Anomaly: An anomaly refers to an observation, node, edge, or substructure that significantly deviates from the majority of the data, often suggesting abnormal or suspicious behavior. In graph settings, anomalies may manifest as unusual connectivity patterns, unexpected feature values, or sudden changes in graph topology. Detecting these outliers is especially critical in cybersecurity, where identifying a single anomalous link or node can prevent broader network breaches.

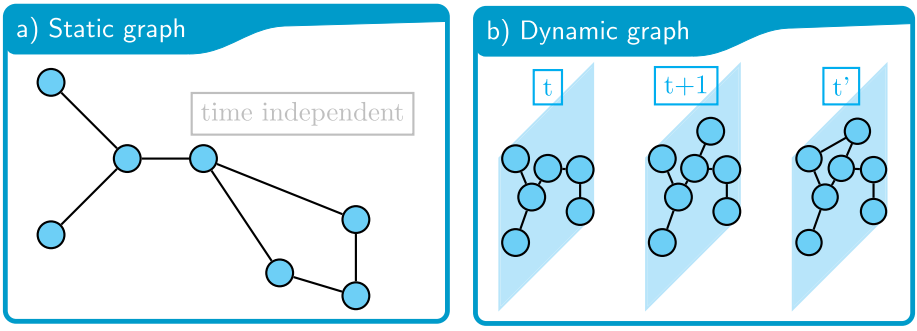

Fig. 2 Comparison of a static graph **a** that remains unchanged and a dynamic graph **b** evolving over time

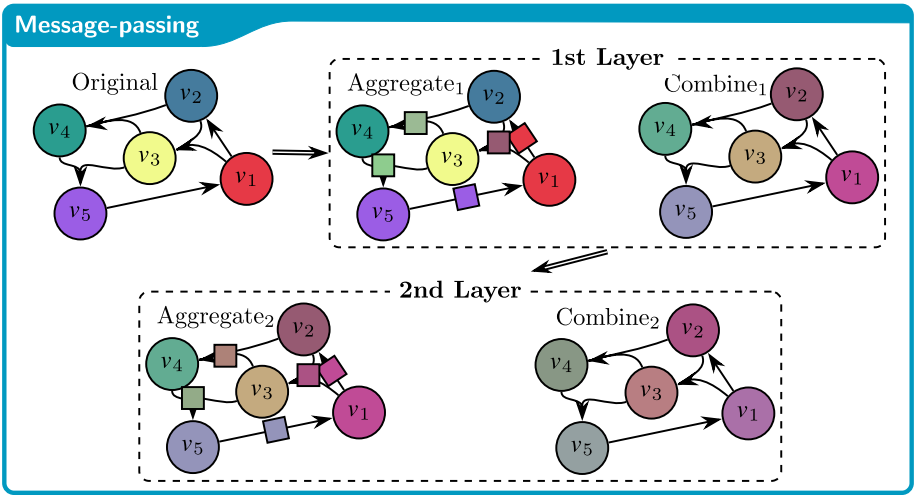

Fig. 3 Two-layer message-passing. Aggregate_1 gathers colored square messages from each neighbor and collapses them into a single summary; Combine_1 produces updated circle embeddings. A second layer (Aggregate_2, Combine_2) repeats the procedure, allowing information to flow across two hops

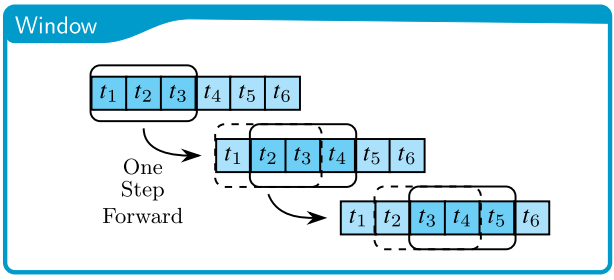

Fig. 4 Sliding temporal window: a fixed-length window (here covering t_1–t_6) advances by one step to produce overlapping graph snapshots

Window: In this work, a window refers to a contiguous time interval over which we aggregate network events into a single graph snapshot. Each window spans a fixed duration and slides forward by a smaller stride, thus capturing evolving relationships in the traffic. Successive windows overlap and share most of their edges and nodes, which preserves temporal continuity. Defining windows in this manner enables GNN-based detectors to flag anomalies that manifest as sudden changes in graph structure or attributes across adjacent windows. Figure 4 shows a visual illustration.

2.2 Levels of graph analysis

Graph-structured data can be analyzed at different granularities, as illustrated in Fig. 5. Node-level approaches focus on classifying or embedding individual vertices; edge-level analysis targets relationships between pairs of nodes, for example by detecting anomalous links. Subgraph-level methods examine localized structures such as communities or motifs, while full-graph approaches treat the entire graph as a single entity for classification or

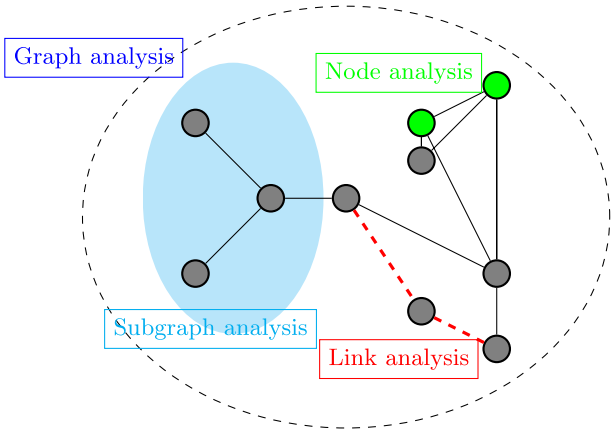

Fig. 5 Illustration of four levels of analysis in GNN-based tasks: node, edge, subgraph, and full-graph

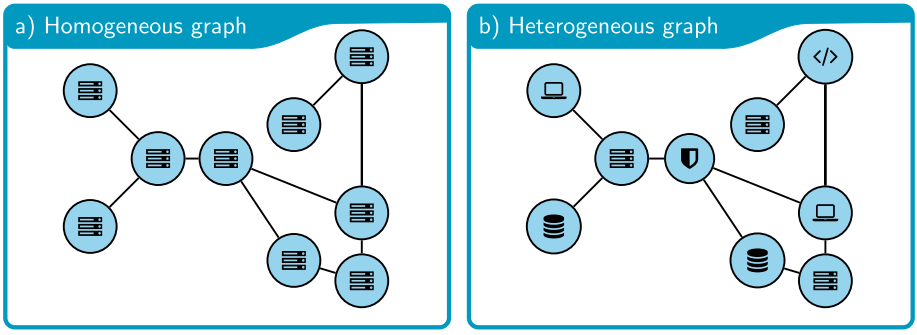

Fig. 6 Homogeneous graph **a** where all nodes represent the same entity type, and heterogeneous graph **b** illustrating multiple entity types

regression. We adopt this four-level taxonomy throughout the review when categorizing anomaly detection tasks and reported results.

2.3 Graph types and representations

Graphs can be classified by their structure and properties, shaping how GNNs process and represent data in cybersecurity applications. Homogeneous graphs assume uniform node and edge types, simplifying analysis by treating all entities equivalently, e.g., servers in a network. While this uniformity streamlines modeling, it may overlook critical distinctions in complex, large-scale environments. In contrast, heterogeneous graphs incorporate diverse node and edge types (e.g., servers, devices, software components, and their interactions), enriching representations to capture intricate relational structures and dependencies. This finer-grained approach provides deeper insights into cyber-threat patterns, as illustrated in Fig. 6. Graphs may also include features–numerical or categorical attributes assigned to nodes or edges–enhancing GNNs' ability to detect subtle anomalies.

2.4 Learning paradigms

The learning strategies used by anomaly detectors can be organized along two orthogonal axes, the inference paradigm and the supervision paradigm. The former specifies how the model is evaluated with respect to the graphs seen during training, while the latter describes the amount and nature of annotation available. Together, these axes yield a compact taxonomy that we will reuse when categorizing the reviewed methods.

2.4.1 Inference paradigms

We distinguish between transductive and inductive inference. In the transductive setting, training and test nodes belong to the same graph, all nodes are visible during training, but only a subset is labeled. The model can therefore exploit the full structural context of test nodes at training time, at the cost of requiring access to the entire graph and retraining or fine-tuning when new nodes appear. In the inductive setting, the model is trained on one or more graphs and later applied to a different graph whose nodes were unseen during training, as illustrated in Fig. 7. Here, the learned message-passing functions must generalize to novel structures, which makes inductive approaches more suitable for dynamic or streaming scenarios, the typical case in cybersecurity.

2.4.2 Supervision paradigm

The supervision axis reflects how much manual labeling is available. In supervised learning, every training node, edge, or graph is associated with a ground-truth label, allowing the model to directly learn a mapping from features to classes. Unsupervised methods dispense with labels and optimize objectives such as reconstruction error or contrastive agreement, while semi-supervised learning assumes that only a small subset of training instances is labeled and propagates this sparse supervision through the graph. Self-supervised approaches synthesize pseudo-labels via pretext tasks (for example, mask recovery or edge prediction) and then transfer the resulting representations to downstream anomaly-detection problems. In practice, these regimes can be combined, giving rise to a rich design space.

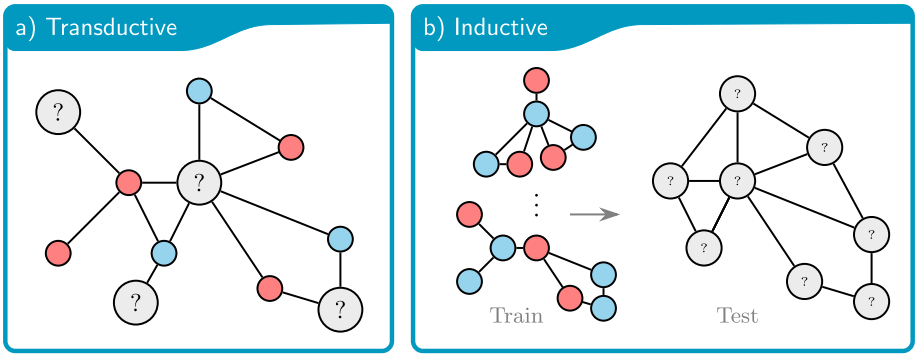

Fig. 7 Node classification under transductive (**a**), a single graph is used for both training and testing, with some node labels hidden, and inductive (**b**), labeled graphs then applied to a separate, unseen graph

2.5 Architectures

In what follows, we introduce the key families of graph neural network models, outlining their fundamental mathematical underpinnings so that their respective strengths and limitations to anomaly detection in cybersecurity become clear.

GCN

GCN generalize convolution to graphs by repeatedly mixing each node's features with those of its neighbors through a normalized neighborhood aggregation rule (Kipf and Welling 2016). After a few layers, each node embedding summarizes both its own attributes and the multi-hop context defined by the adjacency structure. In cybersecurity, GCN are commonly applied to host-communication or flow graphs, where they help surface nodes or subgraphs whose connectivity or traffic patterns deviate from normal behavior, thus supporting intrusion detection and alert triaging.

Most GCN-based detectors were originally designed for transductive scenarios, assuming that the full graph is available during training, but the same propagation scheme can be applied in an inductive way by recomputing embeddings for new nodes or sampled neighborhoods. From a supervision viewpoint, GCN are frequently trained in a (semi-)supervised fashion when labeled benign and malicious instances exist, and can also be combined with reconstruction or contrastive objectives to operate in unsupervised regimes, while remaining relatively lightweight in terms of computational and memory cost.

GAT

GAT extend GCN by replacing fixed, degree normalized averaging with learned attention weights over neighbors (Velickovic et al. 2017). Instead of treating all neighbors equally, a GAT layer assigns a coefficient to each edge that reflects how much a neighbor should influence the updated embedding, allowing the model to emphasize particularly informative nodes and downweight noisy or irrelevant ones. In cybersecurity this is appealing, because anomalous interactions (for example, a low-frequency connection to an unusual host or port) can receive higher attention than the many routine flows that surround them, making covert activity less likely to be smoothed away by simple mean aggregation.

In practice, GAT often exhibit stronger inductive capacity than vanilla GCN, as their attention mechanism adapts naturally to new nodes and partially observed neighborhoods, which is advantageous in dynamic or streaming environments. Regarding supervision, GAT are frequently trained under semi-supervised or self-supervised objectives, attention scores can highlight suspicious relations even when only a small set of labeled incidents is available, though this flexibility comes at increased computational cost due to multi-head attention compared with the single, fixed aggregation used in standard GCN.

Autoencoder-based methods

Autoencoder-based approaches (Kipf and Welling 2016) learn compact latent representations by training an encoder–decoder pair to reconstruct graph structure or node features, rather than predicting labels directly. For graph data, the encoder maps nodes (and sometimes edges) to embeddings, and the decoder attempts to rebuild the original adjacency or feature patterns; nodes, edges, or snapshots with large reconstruction error are then flagged

as potential anomalies. In cybersecurity, this makes autoencoders attractive for modeling "normal" behavior from historical traffic or logs and highlighting flows, hosts, or time windows whose connectivity or attributes deviate sharply from this baseline, even when no ground-truth attack labels are available.

Most graph autoencoders operate in an unsupervised and predominantly transductive regime, since the reconstruction objective is optimized on the observed graph, although encoders built from local operators (for example, shallow GCN) can be reused inductively on new nodes. Their main limitations are the quadratic cost of dense reconstructions and a strong dependence on the representativeness of the training data, which can cause genuine zero-day attacks to be underestimated if the learned manifold is too rigid or biased.

Contrastive approaches

Contrastive graph learning (Hassani and Khasahmadi 2020) trains GNN to produce similar embeddings for different augmented views of the same node, subgraph, or graph, and dissimilar embeddings for unrelated samples. In practice, each training instance is transformed into multiple perturbed versions (for example, via feature masking, edge dropping, or subgraph sampling), and the model optimizes a contrastive objective that maximizes agreement between embeddings of matched views while pushing apart mismatched pairs. For cybersecurity anomaly detection, this self-supervised scheme allows models to learn invariances from unlabeled traffic or log graphs, so that nodes or interactions whose representations remain unstable across augmentations can be flagged as suspicious.

Because positive and negative pairs are generated on the fly from mini-batches, contrastive pipelines are typically inductive and do not require manual labels, which is particularly appealing under zero-day conditions or delayed incident reporting. Their main drawbacks are the sensitivity to the choice of graph augmentations and sampling strategy, and the additional computational overhead introduced by maintaining multiple views per sample, although contrastive objectives can be effectively combined with autoencoders or attention mechanisms to improve robustness on dynamic, large-scale network data.

Figure 8 illustrates four representative architectures: (a) a GCN-based scheme that processes node embeddings through spectral convolutions, (b) a GAT-based approach employing attention weights α_{ij} for neighbor aggregation, (c) an autoencoder-based model that encodes and reconstructs either node features or adjacency information and (d) a contrastive learning pipeline combining multiple augmented views of the same graph.

Transformer-based Transformer-based graph architectures Yun et al. (2019) adapt self-attention to graph data by treating nodes (or subgraphs) as tokens and learning pairwise attention scores, which need not be restricted to the original adjacency. This enables models to capture long-range dependencies and indirect relations that local message-passing GNNs such as GCNs or GATs might overlook. In cybersecurity, such global attention is appealing for uncovering coordinated behavior across distant hosts or services, where attacks manifest as subtle correlations rather than local anomalies.

Most existing graph Transformers are relatively recent, typically rely on unsupervised or self-supervised objectives (for example, mask-and-reconstruct or contrastive alignment between views) to cope with label scarcity, and are usually deployed in transductive settings

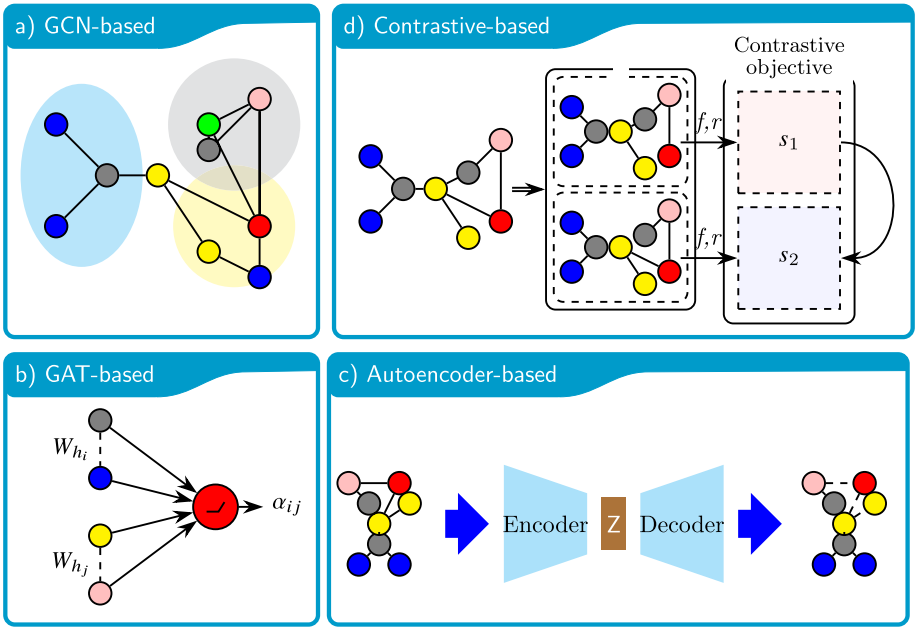

Fig. 8 Comparison of four GNN-based approaches: **a** GCN-based, **b** GAT-based Velickovic et al. (2017), **c** autoencoder-based and **d** contrastive learning. **a** illustrates a GCN-based pipeline, where node embeddings are iteratively updated through spectral-style convolutions. **b** highlights the GAT mechanism, where attention weights (dashed lines) determine how strongly each neighbor influences a node's representation. **c** depicts a graph autoencoder, consisting of an encoder mapping node features to a latent space and a decoder reconstructing adjacency or feature matrices. Lastly, **d** shows a contrastive scheme, with multiple graph views t_1 and t_2 yielding separate embeddings that are encouraged to match under a contrastive objective

where the full graph is available during training. Their main limitation is the quadratic cost of naive self-attention in the number of nodes, which motivates sparse or hierarchical variants to remain scalable on large, dynamic security graphs.

Other approaches

Beyond the canonical GCN-, GAT-, autoencoder- and Transformer-based families, several hybrid architectures combine these building blocks to exploit complementary strengths, for example coupling convolutional encoders with contrastive or reconstruction objectives to improve robustness in anomaly scoring. An emerging line of work explores federated graph learning Zhang et al. (2021), where multiple organizations collaboratively train shared GNN models without centralizing sensitive logs or configurations, which is attractive in cybersecurity settings with strict privacy constraints. Recurrent variants such as Graph-Recurrent Neural Network (RNN), Graph-Gated Recurrent Unit (GRU) and Graph-Graph-Long

Short-Term Memory (LSTM) Li et al. (2017) also appear in dynamic scenarios, allowing node embeddings to evolve over time in response to changes in neighborhood structure or features, albeit at increased implementation and tuning complexity.

2.6 Metrics

Most GNN-based anomaly detectors formulate the task as binary or multi-class detection, and are evaluated with standard machine learning metrics derived from the confusion matrix. For completeness, formal definitions and equations for all metrics mentioned below are collected in Appendix A.

For binary detection, normal vs anomaly, Accuracy measures the overall proportion of correct predictions but quickly becomes misleading under the strong class imbalance typical of intrusion detection. Precision quantifies how many predicted anomalies are truly anomalous, which is relevant for controlling false alerts, while Recall captures how many real attacks are detected, reflecting the risk of missed incidents. The F1 score combines Precision and Recall into a single indicator and is therefore frequently reported in cybersecurity benchmarks. Independent threshold measures such as the area under the ROC curve (AUC) provide an aggregate view across operating points, whereas Matthews Correlation Coefficient (MCC) offers a more balanced summary when classes are highly imbalanced, albeit with less intuitive interpretation. For multi-class settings (for example, when distinguishing several attack categories), variants such as Macro-F1 and Micro-F1 summarize performance across classes, and G-Mean emphasizes balanced detection of minority classes.

A smaller subset of studies treats anomaly detection as a regression problem, for instance when predicting continuous risk scores or resource usage. In these cases, standard error measures such as root mean squared error (RMSE), mean absolute error (MAE), or mean absolute percentage error (MAPE) are used to quantify deviations between predicted and true values, although they provide less direct insight into operational trade-offs. Ultimately, the choice of evaluation metric reflects the system's priorities, such as minimizing missed attacks, controlling false positives, or accurately tracking numerical trends.

3 Methodology for the systematic literature review

This systematic literature review aims to gather a comprehensive and representative collection of studies investigating GNN within the field of cybersecurity, with a special focus on approaches tackling dynamic, time-evolving graphs. To ensure comprehensive coverage of relevant literature, we queried four academic databases: Scopus, IEEE Xplore, Web of Science, and ACM Digital Library. The search was executed using the following Boolean query string and was limited to articles up to and including 2024 in the order listed above:

Table 3 Examples of borderline cases and screening decisions

Case	Ambiguity	Decision	Rule applied
Generic time-series anomaly detection	Temporal modeling is present, but the work is framed as physics.	Exclude	2
Static graph as a fixed schema	A graph is used on a fixed, pre-defined structure (e.g. decision-tree-like attack template).	Exclude	1
ICS faults vs security anomalies	The domain is ICS, but the paper frames faults as reliability/safety.	Include only if threats are explicit and mapped to anomalies	2–3
Security framing but no validation	The contribution is positioned as intrusion/anomaly detection, yet provides no experiments.	Exclude	4
Validation exists but details are insufficient	Experiments are reported, but core details are unclear.	Exclude	5
Synthetic evaluation	The security task is clear and validated, but evaluation relies on simulation or a controlled testbed due to restricted operational data access.	Include	4

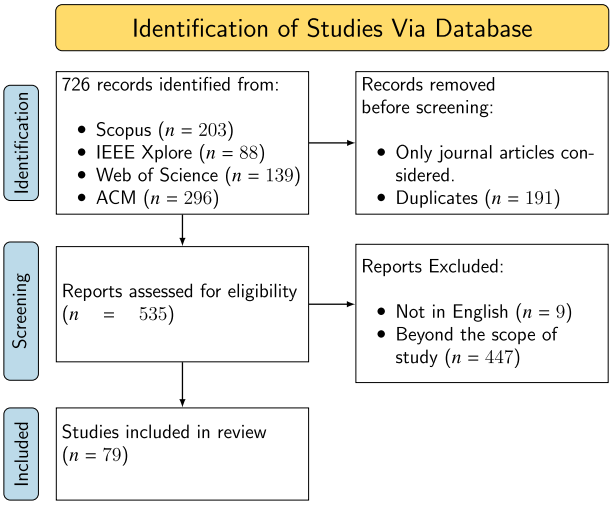

Fig. 9 PRISMA flow diagram for study selection. This diagram illustrates the study selection process in three main phases

```
TITLE-ABS-KEY ( ( "graph neural network" OR GNN OR "graph convolutional
 ↪  network" OR GCN OR GAT OR "graph attention network" OR ( "graph" AND (
 ↪  "transformer" OR "autoencoder" OR "contrastive learning" OR "federated"
 ↪  ) ) ) AND ( cybersecurity OR "intrusion detection" OR "anomaly
 ↪  detection" OR "network security" OR IDS ) AND ( "dynamic graphs" OR
 ↪  "temporal graphs" OR "time series" ) ) AND PUBYEAR < 2025 AND ( LIMIT-TO
 ↪  ( DOCTYPE , "ar" ) ) AND ( LIMIT-TO ( LANGUAGE , "English" ) )

("graph neural network" OR GNN OR "graph convolutional network" OR GCN OR
 ↪  GAT OR "graph attention network" OR ("graph" AND ("transformer" OR "aut
 ↪  oencoder" OR "contrastive learning" OR "federated"))) AND (cybersecurity
 ↪  OR "intrusion detection" OR "anomaly detection" OR "network security" OR
 ↪  IDS) AND ("dynamic graphs" OR "temporal graphs" OR "time series")

ALL=("graph neural network" OR GNN OR "graph convolutional network" OR GCN
 ↪  OR GAT OR "graph attention network"  OR ("graph" AND "autoencoder") OR
 ↪  ("graph" AND "transformer") OR ("graph" AND "contrastive learning") OR
 ↪  ("graph" AND "federated") ) AND ALL=(cybersecurity  OR "intrusion detec
 ↪  tion" OR "anomaly detection"  OR "IoT security" OR "network security" OR
 ↪  IDS) AND ALL=("dynamic graphs" OR "temporal graphs" OR "time series" )

[[All: "graph neural network"] OR [All: gnn] OR [All: "graph convolutional
 ↪  network"] OR [All: gcn] OR [All: gat] OR [All: "graph attention
 ↪  network"] OR [[All: "graph"] AND [[All: "transformer"] OR [All:
 ↪  "autoencoder"] OR [All: "contrastive learning"] OR [All: "federated"]]]]
 ↪  AND [[All: cybersecurity] OR [All: "intrusion detection"] OR [All:
 ↪  "anomaly detection"] OR [All: "network security"] OR [All: ids]] AND
 ↪  [[All: "dynamic graphs"] OR [All: "temporal graphs"] OR [All: "time
 ↪  series"]] AND [E-Publication Date: (01/01/1993 TO 12/31/2024)]
```

The initial search returned 726 records. To determine eligibility, we defined inclusion and exclusion as five screening questions. The questions were applied in two stages: Questions (1)–(3) (mandatory) define the topical scope of this Systematic Literature Review (SLR), while Questions (4)–(5) impose a minimum evidence and reporting threshold. A study was included only if it satisfied all five questions. Borderline cases and how decisions were made are illustrated in Table 3, and the resulting counts are reported in the PRISMA Fig. 9.

1. Does the study explicitly use GNN techniques on temporal/dynamic graph data for anomaly detection? *Rationale: the review targets GNN-based anomaly detection under temporal or dynamic graph settings (e.g., snapshots, evolving edges, temporal signals). Exclude if: the method is non-GNN, or the graph is purely static with no temporal/dynamic component (even if the data are sequential).*
2. Is the problem framing explicitly cybersecurity-oriented? *Rationale: we restrict the review to security use cases, such as intrusion/attack detection, malware/botnet detection, fraud/security monitoring, or security-relevant monitoring in (ICS)/Internet of Things (IoT). Exclude if: the work is generic anomaly detection with no cybersecurity framing (e.g., industrial forecasting anomalies without a threat model).*

Table 4 Data extraction schema used to harmonize the studies by recording the methodological descriptors and evidence needed to answer RQ1–RQ6

Extracted attribute	Operationalization (what was recorded)	Used in
Methodological descriptors		
Bibliographic metadata	Publication year and venue/outlet (descriptive only)	RQ1
Threat focus	Attack typology + operational domain	RQ3–RQ5
Graph temporalization	Dynamic vs. temporal; subcategory; time granularity when specified	RQ3
Learning setup	Supervision paradigm; training scope	RQ2
Architecture family	Backbone family + temporal component	RQ3
Prediction target	primary analysis level; secondary target if explicitly evaluated	RQ4
Evaluation protocol	Datasets used; split strategy when reported; metrics reported	RQ5
Evidence indicators		
Dataset realism	Provenance tag: public benchmark vs. testbed/emulation vs. proprietary logs vs. simulation; presence of native timestamps	RQ5–RQ6
Baseline design	Within-study comparisons (e.g., static vs. temporal, short vs. long horizon); presence of ablations isolating the temporal component	RQ5
Reproducibility	Code availability; reporting of preprocessing/splits/hyperparameters at a usable level	RQ5–RQ6

3. Is the target phenomenon explicitly defined as an anomaly/threat relevant to detection or prevention? *Rationale: the contribution must address anomalies as attacks, intrusions, malicious behaviors, policy violations, or closely related security threats. Exclude if: the task is unrelated (e.g., forecasting or classification without an anomaly/threat objective).*
4. Does the paper report empirical validation in a cybersecurity-relevant setting? *Rationale: some form of evaluation is needed to interpret effectiveness, even when real operational logs are unavailable. Pass if: evaluation is reported on public benchmarks, private/proprietary logs, testbed/emulation, or simulation. Exclude if: the paper is purely conceptual and provides no experimental validation.*
5. Does the paper provide minimum methodological disclosure for meaningful comparison? *Rationale: the review compares methodological choices; at least the core model and evaluation protocol must be described to avoid unverifiable claims. Pass if: the paper reports the core GNN architecture and temporal/dynamic formulation, together with a basic training/evaluation protocol (metrics and experimental setup). Exclude if: these elements are missing or unclear to the point that comparison is not possible.*

This process, detailed in the PRISMA flow diagram (Fig. 9), resulted in 79 studies being selected for the final review. The selection was conducted using the eligibility questions. Records were first screened at title/abstract level, followed by full-text screening for potentially eligible papers. For borderline cases, decisions were resolved by strictly applying the

Table 5 Ref, challenges, and contributions

Refs.	Challenge	Contribution
He et al. (2024)	Cross-domain MTS anomaly detection.	Graph-based cross-domain variance adaptation.
L(y)u et al. (2023)	Hidden contextual anomalies in ICS.	Advanced GNN feature integration technique.
King and Huang (2023)	Anomalous edge detection in an evolving network.	Scalable architecture for temporal graphs.
Yan et al. (2023)	Insufficient device samples impair detection	Graph embedded dual-sequence anomaly detection.
Fu et al. (2024)	Detecting unknown encrypted malicious traffic.	Graph entropy for anomaly detection.
Jiang et al. (2022)	Privacy in distributed surveillance systems.	Secure aggregation protocol with GNN.
Zhang et al. (2024)	Misinformation spread with uncertain propagation.	Convolution with long-tail strategy.
Zhu et al. (2021)	Detecting bot install fraud efficiently.	Enhanced bot fraud detection framework.
Peng et al. (2024)	Interpretable unsupervised social bot detection.	Multi-relational entropy optimization.
Guo et al. (2024)	Overwhelming trace structural–temporal anomaly detection.	GNN-based adaptive trace sampling.
Li et al. (2022)	Unstable network traffic classification.	Graph-based traffic identification.
Sun et al. (2024)	Inefficient root cause localization.	Graph-based multimodal root cause analysis.
Mendoza et al. (2020)	Differentiating bots among legitimate users.	Graph embedding with label propagation.
Zhou et al. (2022)	Ineffective multivariate anomaly detection.	Spatiotemporal GAT-based detection.
Pang et al. (2024)	Ineffective MTS anomaly detection.	Graph-based adaptive detection framework.
Zheng et al. (2024)	Complex multivariate anomaly detection challenges.	GAT with adversarial training.
Wang et al. (2023)	Anomaly detection in multivariate time series.	GCN integration for anomalies.
Qin et al. (2023)	Limited IOT anomaly detection.	Contrastive-based learning framework.
Presekal et al. (2023)	Late-stage cyber attack detection.	GCN-based anomaly identification.
Yang and Yue (2023)	Inefficient feature extraction impedes detection.	Hierarchical graph modeling with hyperspheres.
Zhang et al. (2024)	Ignored propagation changes impair detection.	Graph-based detection using dual deviations.
Akram et al. (2024)	Cyber-physical anomaly detection.	Self-supervised multimodal GNN anomaly detection.
Alrumaih and Alenazi (2024)	Structural vulnerabilities in ICS security.	Graph-based cyberattack detection model.
Liu et al. (2021)	Anomaly detection in dynamic graphs.	Transformers for dynamic graphs.
Zhao and Fink (2024)	Early fault detection in Industrial IoT (IIoT).	Dynamic edge construction, operational modeling.
He et al. (2024)	Cyber-physical anomaly detection.	GAT structure learning detection.
Wang et al. (2023)	Cyber-physical anomaly detection.	Attention-based feature extraction.
Wu et al. (2023)	Enhancing fault detection.	Transformer fusion for heterogeneity.
Benzaïd et al. (2023)	DDoS attack threat.	GAT-based attack detection.

Table 5 (continued)

Refs.	Challenge	Contribution
Zhang et al. (2023)	Ineffective anomaly detection.	GAT-based relationship learning.
Altaf et al. (2024)	Ineffective botnet detection.	Sequential GCN-based botnet identification.
Gao et al. (2023)	Ineffective anomaly detection.	Transformer-based anomaly detection.
Aslan and Choi (2023)	Inefficient authentication methods.	GCN authentication model.
Zhang (2024)	Intrusion detection in ICS.	GAT for spatiotemporal relationships.
Wang et al. (2023)	Ineffective MTS anomaly detection.	Decoupled aggregation, multi-scale attention.
Mir et al. (2024)	Network traffic anomaly detection.	Variational GCN for uncertainty modeling.
Shi et al. (2023)	Correlations in MTS anomaly detection.	GCN-based anomaly detection.
He et al. (2024)	Dependency-aware anomaly detection.	Transformer-based learning for anomalies.
Liu et al. (2023)	Forecasting models vulnerable to attacks.	Hybrid classifier with GCN
Guo et al. (2024)	High-cost IOT anomaly detection.	Energy-efficient GAT.
Wang et al. (2023)	Intrusion detection in remote sensing.	Spatiotemporal GAT for intrusion detection.
Zhao et al. (2024)	Spatiotemporal dependency anomaly detection.	GCN-based learning for anomalies.
Lanko and Makarov (2024)	Underutilized spatial correlations in anomalies.	GAT for anomaly detection.
Ge et al. (2024)	Loss of anomalies in preprocessing.	Contrastive learning.
Liao et al. (2024)	Spatiotemporal CPS anomalies.	GAT-based anomaly detection.
Guan et al. (2024)	Dependency-based MTS anomalies	Autoencoder-based for anomaly detection.
Wen et al. (2024)	Overgeneralization in MTS anomaly detection.	GAT-based anomaly detection.
Zhang et al. (2023)	Privacy-preserving anomaly detection.	Federated GNN for anomaly detection.
Kong et al. (2024)	Inaccurate graph structures in detection.	Graph matching for anomaly detection.
Wang et al. (2024)	Limited early detection and transferability.	Graph-based early detection ensures transferability.
Fan et al. (2024)	Limited spatiotemporal dependency anomaly modeling.	Graph-based anomaly detection enhancement.
Song et al. (2023)	Complex fluctuations hinder anomaly detection.	Explainable GNN ensemble method.
Reddy et al. (2024)	Cryptojacking detection in IOT.	GCN-based time series detection.
Zhang et al. (2024)	Unlabeled cloud anomalies.	Contrastive learning detection.
Jo and Lee (2024)	Limitations in graph-based anomaly detection.	Edge-conditioned GNN for time series.
Liu and Liu (2023)	Malicious attacks in enterprise IOT.	GNN-based intelligent attack detection.
Protogerou et al. (2022)	Security challenges in IOT networks.	Graph-based approach for IOT security.
Fang et al. (2022)	One-dimensional log analysis limits detection.	Contrastive graph classification.
Chen et al. (2023)	Inefficient spatiotemporal anomaly detection.	GAT-based multivariate time series model.

Table 5 (continued)

Refs.	Challenge	Contribution
Li et al. (2022)	Overfitting and inefficiency in anomaly detection.	Autoencoder based learning module.
Zhong et al. (2022)	Dynamic graph stability anomalies.	Dual evolution for nodes and parameters.
Guan et al. (2022)	Spatiotemporal MTS anomalies	GAT anomaly detection model.
Zhao et al. (2024)	Scalability in MTS anomaly detection.	GAT-Informer for anomaly detection.
Huang et al. (2024)	Anomaly detection in multivariate time series.	GAT-based for time-series detection.
Yang et al. (2024)	CPS sensor anomaly detection.	Sparse Autoencoder with Graph Transformer.
Ding et al. (2023)	Multimodal MTS anomaly detection.	GAT and temporal convolution integration.
Peng et al. (2021)	Anomaly detection in networks.	Multi-view model with GAT.
Xie et al. (2024)	Anomaly detection in IOT systems.	Multi-scale feature extraction with GAT.
Zhang et al. (2024)	Spatiotemporal periodicity hampers ICS detection.	Periodic extraction and graph attention.
Tian et al. (2023)	Anomaly detection in IOT systems.	GAT-LSTM for anomaly detection.
Guo et al. (2022)	Ignoring topology degrades Distributed Denial of Service (DDoS) detection.	Combined use of GAT and LSTM.
Liu and Guo (2024)	Neglected dynamic topology impedes detection.	Dynamic integrated GAT.
Zhao et al. (2024)	Overlooked dynamic interdependencies hinder detection.	GNN self-distillation method.
Chiranjeevi and Malathi (2024)	Struggles with complex spatiotemporal anomalies.	Anomaly graph with GCN.
Carletti et al. (2024)	Flow-based approaches neglect structural context.	GCN-based anomaly detection.
Miao et al. (2023)	Anomaly detection in dynamic attributed graphs.	GAT and LSTM.
Fan et al. (2023)	Reconstruction methods inadequately distinguish anomalies.	Time-frequency reconstruction network.
Cao et al. (2022)	Inadequate attack path detection challenge.	GCN and whitelist mitigate DDoS.
Zola et al. (2022)	Imbalanced temporal dynamics hinder detection.	Temporal dissection and graph balancing.

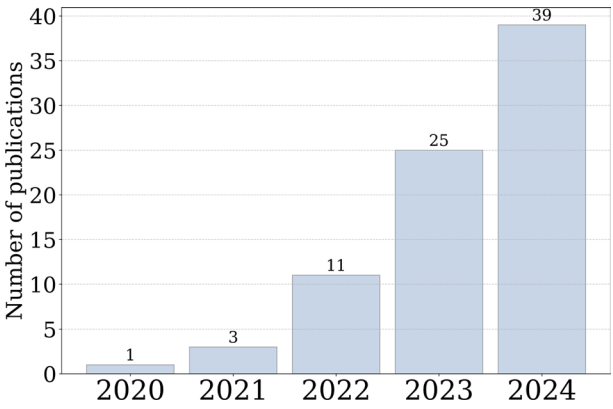

Fig. 10 Annual distribution of reviewed publications, highlighting a clear upward trend

operational definitions of Questions (1)–(5) and documenting the rationale; representative examples are reported in Table 3. The number of records at each step is summarized in Fig. 9.

To reduce the risk of inconsistent decisions, we performed a stability check by re-screening the papers after a washout period, applying exactly the same decision rules (Questions (1)–(5)). When uncertainty remained after full-text inspection (i.e., the paper did not provide enough information to answer a question unambiguously), we applied a conservative rule and excluded the study.

Data extraction was systematically performed using a spreadsheet to capture the descriptors and evidence indicators summarized in Table 4. We recorded a small set of reporting and evidence proxies to contextualize patterns and threats.

Table 5 presents the list of selected articles, including their challenges and main contribution.

4 Research panorama

This section provides an overview of the publication landscape for the 79 studies reviewed addressing Research Question RQ1. Analysis of publication venues reveals a concentration in high-impact journals, predominantly from IEEE Xplore and ACM. The most frequent outlets include: IEEE IoT-J (5% articles), ACM TOIS (4% articles), IEEE Transactions on Industrial Informatics (3% articles), IEEE Transactions on Knowledge and Data Engineering (3% articles).

A quick survey of the publication timeline (Fig. 10) reveals a marked increase in the number of articles over recent years. While there were only one to three works per year prior to 2022, the count jumped to eleven publications in 2022, followed by twenty five in 2023 and thirty nine in 2024. This upward trend suggests rapidly growing interest in dynamic GNN-based anomaly detection–particularly in domains such as cybersecurity, where evolving threats and large-scale data streams make graph-based methods increasingly attractive.

Beyond this high-level panorama, we further examine how methodological choices evolve over time in later results sections, focusing on shifts in supervision paradigms (Fig. 12), architectural families (Fig. 14), and prediction granularity (Fig. 16).

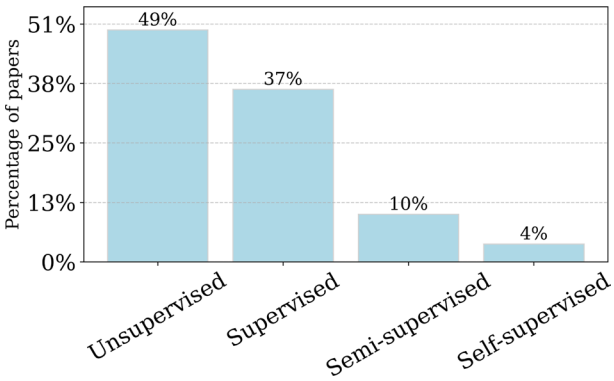

Fig. 11 Distribution of supervision paradigms in the selected studies, showing that unsupervised learning is the most common choice

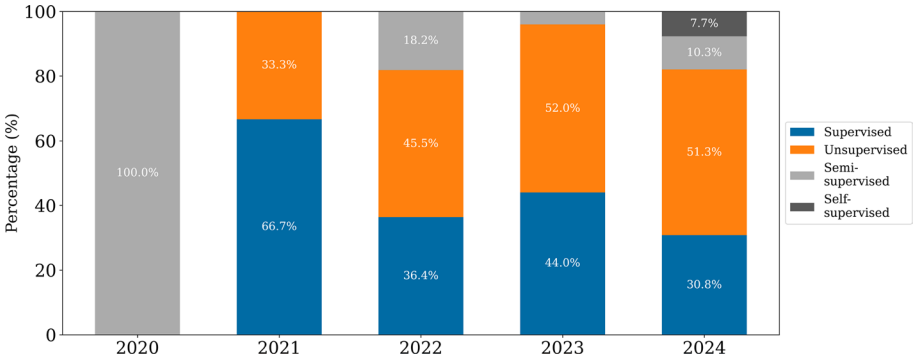

Fig. 12 Evolution of supervision paradigm usage by year, showing increasing reliance on unsupervised learning in recent publications and only sporadic adoption of self-supervised approaches

5 Results

In this section, we explore key GNN architectural trends that use both transductive and inductive inference paradigms for complex graph analysis levels. The rationale for choosing fine or coarse granularity is discussed, highlighting how GNN can adapt over time to changing network behaviors. We also examine how the training scope impacts model robustness and compare GNN-based solutions to other techniques in terms of efficiency, scalability and detection accuracy. Relevant datasets and code availability are noted. Finally, we discuss limitations that hamper direct industrial adoption, outline future directions in interpretability and security, and highlight emerging areas; such as IoT and cyber-physical systems, where GNN-based anomaly detection is poised to make substantial progress.

5.1 RQ2: Learning and inference paradigms

This subsection synthesizes the evidence on how the reviewed studies obtain their training signal and how they define the deployment setting for inference. The goal is to distill what the literature collectively supports under typical constraints (scarce and unstable labels, evolving entities, and interactions).

5.1.1 Supervision paradigms

We start by examining how the selected studies obtain their training signal, distinguishing between supervised, unsupervised, semi-supervised, and self-supervised settings.

The distribution in Fig. 11 indicates that unsupervised training is the most common choice (49%), followed by supervised approaches (37%). Beyond the aggregate split, this reflects two recurring experimental logics. First, many unsupervised papers frame detection as deviation scoring without relying on a stable attack taxonomy, which makes the objective usable even when labels are incomplete or rapidly outdated. Second, supervised papers tend to be benchmark driven. When labels are available, authors optimize directly for classification and report stronger point performance. Many papers also report within-study sensitivity to annotation quality (e.g., label sparsity, label noise, or aging splits) under the same dataset

and protocol. Semi-supervised (10%) studies typically make this trade-off explicit by varying the labeled fraction and reporting how quickly performance saturates as labels increase. Self-supervised (4%) studies usually motivate their choice through within-study operations that compare pretraining enabled representations against the same backbone trained without the pretext objective. Taken together, the analysis suggests that differences between paradigms are often less about a universal accuracy ranking and more about how each paper manages label availability and stability under a fixed experimental setup.

A temporal breakdown in Fig. 12 suggests that unsupervised learning becomes more prominent in recent years: supervised methods dominate the 2021 slice, while unsupervised approaches account for about half of the studies by 2023–2024. Self-supervised designs appear only at the end of the period, indicating early exploration rather than widespread adoption. Since the number of papers varies across years, these proportions should be interpreted as indicative trends.

Some papers also augment the primary supervision setting with auxiliary training mechanisms that refine how the signal is constructed, without changing the overarching paradigm used to categorize them in Fig. 11. Examples include injecting domain rules or heuristics alongside neural components (Guo et al. 2024), multi-stage pipelines that connect representation learning to a downstream decision layer (Presekal et al. 2023; Reddy et al. 2024; Protogerou et al. 2022), contrastive objectives to encourage invariances (Zhang 2024; Fang et al. 2022), and Bayesian/self-labeling components to manage uncertainty (Zhang et al. 2024). These additions are best understood as within-study variations layered on top of the main supervision choice, and therefore they are discussed as extensions rather than separate categories.

5.1.2 Impact of training scope

The evaluation paradigm, specifically the distinction between inductive and transductive settings, can strongly influence the reported performance of GNN-based methods. Under transductive evaluation, models are trained with access to the full graph structure (and, in temporal settings, the complete observed history), which may lead to higher metrics within the corresponding experimental setup. For instance, (Zhang et al. 2023) reports F1-scores of 92.98%, 91.19%, and 87.12% on SWaT, WADI, and HAI, respectively, under its transductive protocol. Likewise, in an industrial control setting, Alrumaih and Alenazi (2024) reports accuracy and F1 close to 99% when exploiting complete graph knowledge, although these values are not directly comparable across studies due to differences in datasets, preprocessing, and evaluation choices.

However, this high performance in transductive settings comes at a cost, as such methods may lack robustness in generalizing to novel, unseen nodes or entirely new scenarios. Conversely, inductive approaches are designed explicitly for generalization, enabling performance in dynamic, evolving environments where unseen data is prevalent. For instance, deep transfer learning has been employed to generalize effectively across network slices–thus attaining an inductive capability while maintaining robust detection performance and achieving over 92% accuracy and accelerating training by at least 61% Benzaïd et al. (2023). Moreover, temporal link prediction experiments have shown that inductive modeling yields approximately a 4% improvement in average precision relative to transductive counterparts

Table 6 Architectural overview, GCN/GAT backbones are the prevailing defaults, and the dominant configurations within families typically align with inductive evaluation and reduced reliance on labels; some studies are counted in multiple families when different architectures are used at different pipeline stages

Category	Representative articles	%	Inference	Supervision
GCN-based	L(y)u et al. (2023); King and Huang (2023); Yan et al. (2023); Fu et al. (2024); Zhang et al. (2024); Zhu et al. (2021); Sun et al. (2024); Wang et al. (2023); Presekal et al. (2023); Yang and Yue (2023); Alrumaih and Alenazi (2024); He et al. (2024); Altaf et al. (2024); Gao et al. (2023); Aslan and Choi (2023); Wang et al. (2023); Mir et al. (2024); Shi et al. (2023); He et al. (2024); Liu et al. (2023); Guan et al. (2024); Zhang et al. (2023); Kong et al. (2024); Fan et al. (2024); Fang et al. (2022); Chen et al. (2023); Zhong et al. (2022); Guan et al. (2022); Zola et al. (2022); Cao et al. (2022); Fan et al. (2023); Chiranjeevi and Malathi (2024); Zhao et al. (2024); Liu and Guo (2024)	32%	Inductive: 68%	Supervised: 44%
GAT-based	He et al. (2024); Zhang et al. (2024); Zhu et al. (2021); Zhou et al. (2022); Zheng et al. (2024); Qin et al. (2023); Zhao and Fink (2024); Wang et al. (2023); Wu et al. (2023); Benzaïd et al. (2023); Zhang et al. (2023); Zhang (2024); Wang et al. (2023); Shi et al. (2023); Guo et al. (2024); Wang et al. (2023); Zhao et al. (2024); Lanko and Makarov (2024); Ge et al. (2024); Liao et al. (2024); Guan et al. (2024); Wen et al. (2024); Zhang et al. (2023); Wang et al. (2024); Fan et al. (2024); Song et al. (2023); Zhang et al. (2024); Chen et al. (2023); Guan et al. (2022); Miao et al. (2023); Guo et al. (2022); Tian et al. (2023); Zhang et al. (2024); Xie et al. (2024); Peng et al. (2021); Ding et al. (2023)	32%	Inductive: 63%	Unsupervised: 64%
Transformer-based	Qin et al. (2023); Liu et al. (2021); Wu et al. (2023); Gao et al. (2023); Wang et al. (2023); Wen et al. (2024); Yang et al. (2024)	6%	Transductive: 57%	Unsupervised: 86%
Autoencoder-based	Zheng et al. (2024); Wang et al. (2023); Akram et al. (2024); Alrumaih and Alenazi (2024); Lanko and Makarov (2024); Fan et al. (2024); Chen et al. (2023); Li et al. (2022)	7%	Inductive: 62%	Unsupervised: 75%
Contrastive learning	Zhang et al. (2024); Qin et al. (2023); Ge et al. (2024); Zhang et al. (2024); Fang et al. (2022)	4%	Inductive: 60%	Unsupervised: 60%
Other approaches	Jiang et al. (2022); Peng et al. (2024); Zhou et al. (2022); Zheng et al. (2024); Zhao et al. (2024); Zhang et al. (2023); Carletti et al. (2024); Tian et al. (2023); Ding et al. (2023); Yang et al. (2024); Huang et al. (2024); Zhao et al. (2024); Guo et al. (2024); Li et al. (2022); Mendoza et al. (2020); Pang et al. (2024); Zhang et al. (2024); Jo and Lee (2024); Liu and Liu (2023); Protogerou et al. (2022)	17%	Inductive: 72%	Unsupervised: 60%

King and Huang (2023), demonstrating the practical benefits of integrating inductive methods when dealing with temporal dynamics Zhang et al. (2024).

Ultimately, the choice between the inductive and transductive paradigms profoundly influences both the reported numerical metrics and the practical applicability of the methods. Although transductive settings optimize performance within known datasets, inductive methods ensure broader adaptability to evolving and dynamic conditions, highlighting an essential trade-off researchers must consider carefully when designing GNN-based systems.

5.2 RQ3: Architectures and temporal integration

This subsection consolidates the evidence on two linked design choices, which GNN encoder families are used as the structural backbone, and how time is incorporated when the underlying process evolves. Rather than treating the literature as a single benchmark, the emphasis is on recurring architectural patterns and on within-study evidence for the role of temporal modeling.

5.2.1 Architectural trends

We next examine which architectures are adopted to build the graph encoder in the reviewed papers, complementing the background overview in Sect. 2.5. The goal is not to rank architectures across experimental setups, but to map which families are most frequently used and how they are typically paired with inference scope and supervision choices within each family.

Table 6 summarizes the main architectural families found. The reported percentages are descriptive indicators of popularity and are not mutually exclusive, since some studies combine more than one backbone (e.g., one module for feature extraction and another for refinement). Two families dominate the landscape, GCN and GAT designs, each accounting for 32% of the reviewed papers. This prevalence is partly explained by practicality both are comparatively simple message-passing pipelines that serve as strong defaults, require

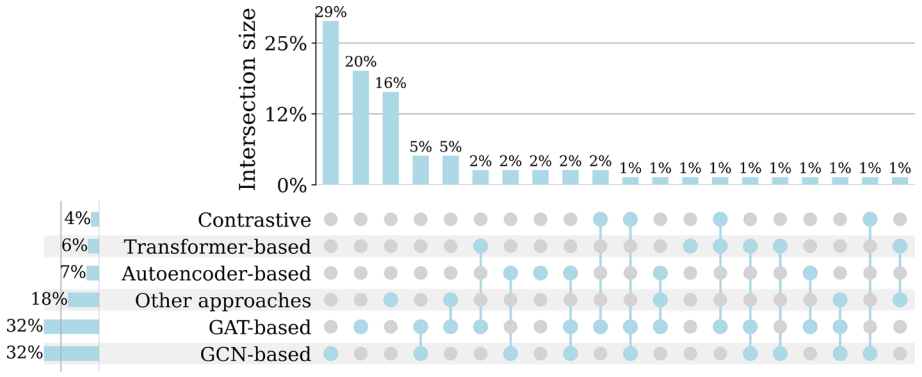

Fig. 13 UpSet plot illustrating the intersection of major GNN architectural families across the reviewed articles. The horizontal bars on the left quantify the total number of articles tagged with each family (including those that also appear in other sets), while the vertical bars at the top measure the size of each specific intersection (filled dots indicate which families are combined). Percentages on the vertical bars refer to the fraction of the total 79 articles; percentages on the horizontal bars refer to the fraction of all category-tags

limited architectural engineering, and are readily available in mainstream libraries, making them attractive starting points for new anomaly-detection pipelines. The remaining share is distributed across transformer-based (6%), autoencoder-based (7%), contrastive learning (4%), and a heterogeneous set of other approaches (17%). The latter aggregates diverse, typically lightweight graph-learning choices that do not fit the major encoder families. These include random-walk or spectral feature constructions, matrix-factorization or embedding baselines, and classical/statistical detectors applied to derived graph representations. Their collective weight reflects breadth rather than a single dominant alternative.

The breakdown in Table 6 also highlights how architectural choices tend to co-occur with evaluation scope and supervision. Inductive inference is prevalent across most families (60–72%), indicating that many works explicitly target generalization to unseen instances rather than optimizing for a fixed, fully observed graph snapshot. Transformer-based models are the main exception, where transductive evaluation is more common (57%), suggesting that these designs are frequently assessed under full-graph access.

Regarding supervision, the split in Table 6 suggests two different usage patterns. GCN-based pipelines more often appear in label-driven designs (44%), which is consistent with how they are typically employed, a relatively simple, stable structure that is easy to optimize when labels are provided, making it a natural choice for supervised intrusion detection formulations. In contrast, GAT-based studies are more frequently trained without labels (64%), reflecting their common role in settings where authors rely on neighbor-importance weighting to build anomaly scores from structural cues rather than from explicit attack annotations. Smaller families also tend to favor label-free objectives because their core mechanisms (reconstruction or agreement across views) provide a direct training signal in the absence of ground truth.

A temporal view in Fig. 14 complements the aggregate distribution by showing how architectural choices diversify over time. Early years are dominated by the catch-all category, while later years exhibit a more balanced mix where GCN- and GAT-based models remain common but coexist with a wider range of alternatives. This trend supports the interpretation that the field is gradually exploring a broader design space rather than converging on a single architecture.

The intersection analysis in Fig. 13 further indicates that most studies adopt a single dominant backbone, while multi-family combinations appear less frequently. This is a plausible outcome for applied anomaly detection, combining strong blocks typically increases computational cost and engineering overhead (more hyperparameters and more complex training schedules), while the incremental improvements are not always clear or consistent. When combinations occur, they often reflect modular pipelines (commonly GCN or GAT) is paired with an auxiliary learning principle, such as reconstruction or contrastive objectives. This also explains why contrastive learning rarely appears as a stand-alone architectural family in practice, it is commonly implemented as a training strategy layered on top of an underlying encoder, and is therefore reported in conjunction with the architecture that performs message-passing.

5.2.2 Adaptive models over time

We analyze how the reviewed studies incorporate time into graph representations. The taxonomy in Table 7 separates dynamic formulations, where the topology itself changes, from

Table 7 Taxonomy of graph usage

Main category	Subcategory	Representative articles	Architectures	Supervision paradigm
Dynamic graphs	General or evolving snapshots	He et al. (2024); L(y)u et al. (2023); Yan et al. (2023); Fu et al. (2024); Jiang et al. (2022); Guo et al. (2024); Zhou et al. (2022); Pang et al. (2024); Zheng et al. (2024); Wang et al. (2023); Qin et al. (2023); Presekal et al. (2023); Yang and Yue (2023); Zhang et al. (2024); Akram et al. (2024); Liu et al. (2021); Zhao and Fink (2024); Benzaïd et al. (2023); Zhang (2024); Mir et al. (2024); He et al. (2024); Guo et al. (2024); Liao et al. (2024); Zhang et al. (2023); Reddy et al. (2024); Jo and Lee (2024); Chen et al. (2023); Zhong et al. (2022); Cao et al. (2022); Miao et al. (2023); Carletti et al. (2024); Chiranjeevi and Malathi (2024); Zhao et al. (2024); Liu and Guo (2024); Guo et al. (2022); Tian et al. (2023); Xie et al. (2024); Li et al. (2022)	GCN 33% GAT 29% Transformers 4% Autoencoders 9% Contrastive 3%	Unsupervised 52% Supervised 31% Semi-supervised 13% Self-supervised 4%
	Temporally labeled	Liu et al. (2023); Zhao et al. (2024); Ge et al. (2024); Zhao et al. (2024)		
	Pattern-based	Kong et al. (2024); Huang et al. (2024)		
	Window-based	Zhu et al. (2021); Sun et al. (2024); Protogerou et al. (2022)		
	Constructed from temporal data	Peng et al. (2024); Guan et al. (2022); Li et al. (2022); Alrumaih and Alenazi (2024); Wang et al. (2023)		
Temporal graphs	General (time-ordered edges)	King and Huang (2023); He et al. (2024); Wang et al. (2023); Wu et al. (2023); Zhang et al. (2023); Aslan and Choi (2023); Shi et al. (2023); Lanko and Makarov (2024); Fan et al. (2023); Zhang et al. (2024); Peng et al. (2021); Ding et al. (2023); Gao et al. (2023); Yang et al. (2024)	GCN 28% GAT 40% Transformers 10% Autoencoders 5% Contrastive 8%	Unsupervised 44% Supervised 48% Semi-supervised 4% Self-supervised 4%
	Specialized (cascades, multi-edge, correlatives, relationship modeling)	Zhang et al. (2024); Mendoza et al. (2020); Altaf et al. (2024); Song et al. (2023); Liu and Liu (2023); Zola et al. (2022); Wang et al. (2023); Guan et al. (2024); Wen et al. (2024); Fan et al. (2024); Zhang et al. (2024); Wang et al. (2024); Fang et al. (2022)		

Dynamic formulations are more common and they typically pair lightweight message passing (GCN/GAT) with predominantly unsupervised objectives

temporal formulations, where interactions are time-stamped while the underlying node set and connectivity assumptions remain comparatively stable. Within our corpus, dynamic graphs are more common than temporal graphs, suggesting that many authors prefer to model evolution explicitly through structural updates rather than treating time only as an edge attribute.

Dynamic graphs in Table 7 are divided through five mechanisms that mainly differ in how they discretize change. General or evolving snapshots build a sequence of graphs and update connectivity as new observations arrive, capturing changes such as newly active communication paths or disappearing links (Presekal et al. 2023). Temporally labeled variants store explicit creation times for nodes or edges to support age-sensitive reasoning. Pattern-based approaches add edges only when a predefined structure is observed, prioritizing semantically meaningful increments over raw volume. Window based designs constrain the time scale by maintaining a sliding horizon, and constructed from temporal data graphs derive topology from time series (e.g., correlations) at each step, turning evolving signals into an evolving adjacency.

These design choices help explain the encoder mix reported for the dynamic branch. Snapshot-style pipelines repeatedly apply message-passing to a sequence of discretized topologies, which aligns well with the simplicity and efficiency of GCN aggregation (33%) relative to GAT (29%). In practice, dynamic graph models often introduce a second temporal component (explicit recurrence, convolution over time, or attention across snapshots) to represent ordering, while keeping the snapshot encoder lightweight (Wang et al. 2023; Liu et al. 2021; Zhao and Fink 2024). This division of labor is also visible in the supervision split. Over half of dynamic studies use unsupervised objectives (52%), indicating that many contributions treat temporal adaptation as a way to maintain sensitivity under drift without relying on stable labels.

Temporal graphs take a complementary position, interactions are time-stamped, and the learning task is to interpret when edges occur (and in what sequence) rather than to rebuild topology at each step. The general subfamily models time-ordered events directly, while specialized variants enrich the event stream with additional structure (e.g., typed relations or relationship modeling) Zhang et al. (2024); Wen et al. (2024); Zhang et al. (2024). This framing naturally increases the importance of selectively weighting interactions, which is consistent with the higher share of GAT-based encoders in the temporal branch (40%) in Table 7. Compared with the dynamic branch, supervision is closer to balanced (unsupervised 44% vs. supervised 48%), reflecting that temporal-edge formulations are often evaluated in settings where labeled events are available, while still retaining a substantial label-free segment.

A practical boundary case is that some works approximate time by aggregation (e.g., collapsing events into snapshot windows or correlations), which simplifies modeling but can hide fine-grained temporal order (Zhu et al. 2021; Peng et al. 2024). In contrast, datasets without native timestamps can be retrofitted with synthetic chronologies to enable time-aware evaluation, as shown by approaches that impose artificial ordering to adapt legacy benchmarks to dynamic pipelines (Mir et al. 2024). Both choices illustrate that time-awareness is not binary, it spans a spectrum from coarse discretization to event-level modeling.

Table 8 links attack focus to the modeling decisions most frequently adopted within each threat family, providing context for the architectural and supervision trends discussed in Table 7. The most visible pattern is the strong concentration on FDI/spoofing and

Table 8 Attack-typology focus and dominant design choices

Attack typology	*n*	Most common supervision	Most common architecture family	Dynamic vs. Temporal
DDoS / flooding / volumetric	4%	Supervised: 100%	GAT-based: 100%	Dynamic ≈ 56%
Scanning / probing / brute-force	1%	Unsupervised: 100%	Autoencoder-based: 100%	Dynamic ≈ 100%
Botnets / IoT malware	6%	Supervised: 60%	GCN-based: 60%	Dynamic ≈ 72%
Malware / ransomware / cryptojacking	4%	Semi-supervised: 66%	GCN-based: 66%	Dynamic ≈ 62%
Lateral movement / APT / insider-like	5%	Unsupervised: 50%	GAT-based: 50%	Dynamic ≈ 64%
FDI / spoofing / control manipulation	55%	Unsupervised: 56%	GAT-based: 45%; GCN-based: 43%	Dynamic ≈ 62%
Web/app specific	1%	Supervised: 100%	GAT-based: 100%	Dynamic ≈ 100%
Social bots / platform abuse	8%	Unsupervised: 50%	GCN-based: 50%; GAT-based: 50%	Dynamic ≈ 62%
Multi-attack (IDS benchmarks / mixed families)	16%	Unsupervised: 69%	GCN-based: 61%	Dynamic ≈ 60%

Control-manipulation settings dominate and are most often addressed with unsupervised, dynamic-graph pipelines

control-manipulation scenarios, which largely explains why unsupervised objectives and dynamic formulations are prevalent. These settings often arise from telemetry where labels are incomplete, and topology is typically reconstructed or updated over time. Conversely, categories such as DDoS/flooding or web/app-specific attacks appear in only a handful of studies, so the most common choices there should be read as descriptive rather than definitive. When temporal context yields gains, authors usually justify it by ablations against their own static or short-horizon baselines under the same protocol (e.g., improvements when enabling longer-range memory or attention), and these benefits are most consistent in multi-step threats where deviations accumulate over time rather than as isolated spikes. In short, Table 8 clarifies what is being detected so that temporal comparisons can be interpreted in the right operational context, without conflating the results across heterogeneous benchmarks.

To avoid conflating results across heterogeneous datasets, we interpret empirical impact primarily through within-study comparisons. Table 9 summarizes representative systems and how each paper motivates temporal modeling by benchmarking against its own static or short-horizon baselines. In these works, reported gains are typically attributed to the ability to retain context over longer horizons. For example, Jo and Lee (2024) reports a higher macro-F1 when temporal retention is introduced, Liao et al. (2024) reports substantial improvements when long-range attention is enabled, and King and Huang (2023) shows a large precision increase when recurrent link prediction is added. Although magnitudes vary by task and dataset, the consistent within-study pattern is that temporal memory improves detection when attacks unfold over multiple steps rather than as isolated spikes.

The systems in Table 9 cover both slow and fast attack dynamics, which helps to clarify when the temporal context matters most. Long-term or mixed memory tends to be emphasized in scenarios where deviations accumulate gradually, such as multi-stage enterprise intrusions (King and Huang 2023) or slowly injected perturbations in Cyber Physical System (CPS) and ICS settings (Presekal et al. 2023; Wang et al. 2023). Short-term memory

Table 9 Comparative analysis of key temporal-modeling characteristics of representative graph-based anomaly detection systems

Paper	Year	Temporal Memory	Threat focus	Detection Improvement	Scalability	Lightweight Optimization	Datasets
King and Huang (2023)	2023	Mixed	Lateral Movement	High	Parallelizes over workers	Decoupling layers	Email/social-media traffic (Public)
Wang et al. (2023)	2023	Short-term	Fault diagnosis	Slight	Quadratic in N	Multiscale wavelets	Telemetry (Public)
Presekal et al. (2023)	2023	Long-term	FDI / spoofing control manipulation	Slight	–	Adaptive sampling	CPS attacks (Own)
Liu et al. (2021)	2023	Mixed	Generic	Medium	Linear in edges and the window size	Sampling strategy	Email/social-media traffic (Public)
Wang et al. (2023)	2023	Mixed	Generic	High	–	Disentangled into fixed vs. dynamic components	Telemetry (Public)
Shi et al. (2023)	2023	Mixed	ICS attacks	Medium	No, roughly quadrupled runtime	kNN feature graph per window	Telemetry (Public)
Miao et al. (2023)	2023	Mixed	Network-based attacks	Slight	Edge-dependent per snapshot	Low-dimensional node embeddings	Networks Attacks (Public)
Zheng et al. (2024)	2024	Mixed	Generic	High	–	Multiscale 1D convolutions	Telemetry (Public)
Zhao and Fink (2024)	2024	Short-term	Generic	Medium	No, edge construction is quadratic	–	Telemetry (Own)
Benzaïd et al. (2023)	2024	Mixed	DDoS	Slight	Tolerates a big sliding-window	Pre-extracts local temporal features	Network Traffic (Own)
Lanko and Makarov (2024)	2024	Long-term	Generic	Medium	–	Reduced diffusion steps.	Telemetry (Public)
Liao et al. (2024)	2024	Short-term	Generic	Slight	Fully connected scoring; sparse after pruning	Top-k sparsification; batch score pooling	Telemetry (Public)
Zhang et al. (2023)	2024	Mixed	Generic	High	Linearithmic	Sliding-window in a cyclic fashion	Network Traffic (Public)
Jo and Lee (2024)	2024	Short-term	Generic Server and CPS	Medium	Top-k sparse adjacency	–	Telemetry (Public)
Liu and Guo (2024)	2024	Long-term	Generic	Slight	Propose streaming size	Line-graph transformation	Network Traffic (Public)

Table 9 (continued)

Paper	Year	Temporal Memory	Threat focus	Detection Improvement	Scalability	Lightweight Optimization	Datasets
Wen et al. (2024)	2024	Mixed	Generic	Slight	No, high computational complexity	–	Telemetry (Public)
Fan et al. (2024)	2024	Long-term	Generic	Medium	Parallelizable	Small gating subnetwork	Telemetry (Public)
Zhao et al. (2024)	2024	Mixed	Generic	Medium	Linearithmic	Using self-attention distilling	Telemetry (Public)
Zhao et al. (2024)	2024	Short-term	Generic	Slight	Cost grows with window length	Multiscale convolutions	Telemetry (Public)

Showing that adding temporal memory most often improves detection within-study comparisons, but it typically increases computational cost

Table 10 Graph analysis levels in the reviewed studies

Level	Subcategory	Articles
Nodes	Node-level scoring on telemetry variables	He et al. (2024); L(y)u et al. (2023); Yan et al. (2023); Zhou et al. (2022); Wang et al. (2023); Gao et al. (2023); Wang et al. (2023); Shi et al. (2023); Liu et al. (2023); Lanko and Makarov (2024); Li et al. (2022); Guan et al. (2022); Zhao et al. (2024); Huang et al. (2024); Ding et al. (2023); Xie et al. (2024); Tian et al. (2023); Miao et al. (2023); Fan et al. (2023)
	Node-level scoring on network entities	Presekal et al. (2023); Wang et al. (2023); Benzaïd et al. (2023); Peng et al. (2021); Zhang et al. (2024); Carletti et al. (2024); Zola et al. (2022)
	Specialized cases	Jiang et al. (2022); Zhang et al. (2024); Sun et al. (2024); Chen et al. (2023); Chiranjeevi and Malathi (2024)
Edges	General edge-level scoring	King and Huang (2023); Fu et al. (2024); Zhu et al. (2021); Guo et al. (2024); Altaf et al. (2024)
Graphs	General graph-level prediction	Aslan and Choi (2023); Fang et al. (2022)
Nodes and Edges	Joint node–edge scoring on telemetry graphs	Pang et al. (2024); Zheng et al. (2024); Qin et al. (2023); Yang and Yue (2023); Zhang et al. (2024); Akram et al. (2024); Zhao and Fink (2024); He et al. (2024); Zhang et al. (2023); He et al. (2024); Ge et al. (2024); Liao et al. (2024); Guan et al. (2024); Wen et al. (2024); Zhang et al. (2023); Kong et al. (2024); Song et al. (2023); Reddy et al. (2024); Zhang et al. (2024); Jo and Lee (2024); Liu and Liu (2023); Yang et al. (2024); Zhao et al. (2024)
	Joint node–edge scoring on interaction graphs	Li et al. (2022); Alrumaih and Alenazi (2024); Zhang (2024); Mir et al. (2024); Wang et al. (2024); Fan et al. (2024); Protogerou et al. (2022); Zhong et al. (2022); Guo et al. (2022); Liu and Guo (2024); Cao et al. (2022)
Nodes and Subgraphs	General nodes and subgraphs	Peng et al. (2024); Mendoza et al. (2020); Liu et al. (2021); Wu et al. (2023); Guo et al. (2024); Wang et al. (2023); Zhao et al. (2024)

Most works target node-level decisions or jointly model nodes and edges, whereas edge and graph-level predictions appear only in a small fraction

is more commonly positioned for sudden deviations (e.g., abrupt traffic changes), where recent history is sufficient to separate anomalies from benign fluctuations (Jo and Lee 2024). Rather than treating more memory as universally better, the within-study comparisons in these papers typically align memory design with the expected temporal footprint of the threat.

The same table also makes the engineering trade-off explicit. Temporal context increases cost (in parameters, neighborhood size, or window length), so most works introduce targeted optimizations to keep the approach deployable. Examples include separating fixed from dynamic components (Wang et al. 2023), compressing temporal signals (Shi et al. 2023), pruning benign subsequences before full processing (Liao et al. 2024), and bounding neighborhoods through sampling or sparsification (Liu et al. 2021; Jo and Lee 2024). These strategies indicate that the field is converging on a practical recipe: add temporal retention where it improves detection in a controlled ablation, then manage the overhead through sparsity, buffering, or decoupled modeling.

5.3 RQ4: Graph construction and analysis granularity

This subsection synthesizes how the reviewed works define graph objects and prediction targets, and how these choices shape what anomalies can be localized and explained. The emphasis is on what the literature consistently supports about granularity choices and on where evidence remains sparse.

5.3.1 Graph analysis levels

In this part, we examine the primary prediction granularity adopted by the surveyed studies, using the analysis-level taxonomy introduced in Sect. 2.2. The goal is to clarify what each method ultimately predicts and evaluates (nodes, edges, whole graphs, or mixed targets), which in turn influences how anomalies are localized and acted upon in practice.

The distribution across levels is summarized in Fig. 15. Two families dominate, node-level outputs and joint node–edge targets. This prevalence aligns with operational requirements in cybersecurity, where analysts typically need to identify which entity is compromised (a host, sensor, service, or variable) and, increasingly, which interaction supports the alert (a flow, dependency, or communication). Edge-only objectives appear in fewer works, mainly when interactions are the natural unit of analysis (e.g., temporal links or flow relations), while graph-level prediction remains uncommon because many security scenarios require fine-grained localization rather than a single label for an entire network snapshot.

A more detailed view is provided in Table 10, which groups papers by recurring modeling patterns within each level. For node-level analysis, two consistent semantics emerge, node-wise scoring on telemetry variables (common in multivariate time-series settings) and node-wise scoring on network entities (hosts, devices, or infrastructure components). A small set of specialized cases covers less frequent node semantics, such as multimodal anomaly detection (Chen et al. 2023) or oriented surveillance formulations (Chiranjeevi and Malathi 2024). The joint node–edge level captures studies that explicitly model both entity states and their relations, either on telemetry graphs (variable dependencies changing over time) or on interaction graphs (communication-driven dependencies). Finally, the node and subgraph level collects works where the target includes community or substructure-oriented signals, which is useful when malicious behavior is expressed as coordinated activity rather than isolated entities.

Changes over time are shown in Fig. 16. Early works are more frequently node-centric, while recent studies more often adopt joint node–edge formulations. This shift is consistent with the growing emphasis on modeling dependencies that evolve (e.g., temporal correlations among variables or interaction patterns among entities), and with the availability of mature libraries that make relational architectures easier to implement and scale. Edge and graph-level objectives remain niche over the same period, reflecting their stronger assumptions. Edge-centric setups require stable level interaction supervision or strong proxy objectives, and graph-level labels are less aligned with the localization needs of many security monitoring pipelines.

5.3.2 Rationale for granularity

We discuss the role of granularity and how it affects anomaly detection, computational cost, and interpretability. Granularity here refers to the level of detail at which entities and interactions are represented in the graph, both in space (what a node or an edge corresponds to) and in time (over which interval observations are aggregated).

At the node level, many studies in industrial control systems represent individual sensors or control tags as nodes (He et al. 2024; Zhou et al. 2022; Pang et al. 2024; Yang and Yue 2023; Shi et al. 2023). This choice allows models to localize anomalies at specific devices and to inspect which neighboring sensors contribute to an alarm. In network traffic monitoring, graphs often use hosts or IP addresses as nodes, with edges induced by observed flows or connections (L(y)u et al. 2023; Li et al. 2022; Sun et al. 2024; Mendoza et al. 2020). Social and bot detection datasets typically model user accounts as nodes and follow, mention, or interaction links as edges, which helps identify suspicious communities or influential accounts (Zhang et al. 2024; Alrumaih and Alenazi 2024; Liu et al. 2021; Peng et al. 2024). Some works construct hierarchical graphs that group related devices or signals into higher level units such as services or subnets while still preserving fine grained structure inside each group (Yan et al. 2023; He et al. 2024; Wu et al. 2023).

Temporal resolution provides a second dimension of granularity. Several approaches build graphs at the level of individual events or short time steps, capturing rapid changes in behavior, but increasing the number of snapshots to process (King and Huang 2023; Zhao and Fink 2024). Others aggregate observations over longer windows or construct evolving snapshots of the system state, which reduces computational cost and can stabilize predictions but risks averaging out short lived attacks or mixing different regimes inside a single graph (Wang et al. 2023; Mir et al. 2024). Multi scale schemes combine both views, with coarse representations tracking slow trends and fine grained graphs focusing on local deviations (Qin et al. 2023; Akram et al. 2024; Wang et al. 2023; Liu et al. 2023; Zhao et al. 2024).

In the surveyed literature, these design choices are rarely explored in a systematic way. Some works explicitly study how different node or temporal resolutions affect performance (Zhang et al. 2023; Reddy et al. 2024; Zhang et al. 2023; Fan et al. 2024; Guan et al. 2022; Wen et al. 2024; Jo and Lee 2024; Liu and Liu 2023), but in most cases granularity is fixed by the dataset definition or by implementation convenience. Therefore, it remains unclear how much of the reported performance gains can be attributed to the modeling capacity of the GNN itself and how much depends on the particular resolution at which nodes, edges, and time are defined. More detailed analysis of node, edge, and temporal granularity would

Table 11 Top referenced methods in comparative evaluations

Method	Type of method	Percentage of mentions
OmniAnomaly	Traditional (LSTM-based)	26%
GDN	Graph-based (Temporal)	25%
MTAD-GAT	Graph-based (Temporal)	12%
GAT	Graph-based (Static)	13%
USAD	Traditional (Autoencoder)	7%
DAGMM	Traditional (Autoencoder)	6%
MAD-GAN	Traditional (GAN-based)	6%
TranAD	Traditional (Transformers)	6%
GraphSAGE	Graph-based (Static)	5%
GCN	Graph-based (Static)	5%

help future work balance detection accuracy, computational efficiency, and interpretability in real deployments.

5.4 RQ5: Evaluation practice and potential biases

This subsection summarizes how the reviewed works evaluate their proposals, focusing on three recurring dimensions: baseline selection, dataset choice, and metric reporting. The aim is to identify what the literature reliably supports about current evaluation practice and where common biases still limit comparability and deployment relevance.

5.4.1 Validation strategies

In this section, we examine the benchmark methods that the primary studies in this SLR employ as baselines to evaluate their anomaly detection approaches.

Table 11 shows that OmniAnomaly and Graph Deviation Network (GDN) together account for more than half of all mentions in comparative tables. OmniAnomaly is not a graph model, but it became an early and well documented baseline for multivariate time series anomaly detection on benchmarks such as SWaT, SMD, and SMAP. Its public implementation and established performance make it a natural choice when authors want to show that their proposal improves on a strong sequential reference without explicit topology. GDN, in contrast, already combines graph convolutions with temporal convolutions in a compact design that matches the sensor networks found in many industrial control and IoT datasets. New temporal graph architectures are therefore often validated between these two reference points: OmniAnomaly, which tests whether relational inductive biases add value at all, and GDN, which acts as a lightweight graph-based baseline.

The stacked bar chart in Fig. 17 illustrates how these baselines are distributed across architectural families. Studies that model dynamic graphs (solid segments) are more common than those that use temporal graphs with a fixed topology (hatched bars) in almost every family, which indicates a preference for baselines that adapt edge structure over time. In the dynamic graph columns, GDN and MTAD-GAT concentrate most of the mentions for GCN and GAT style architectures, while transformer and contrastive frameworks rely more heavily on OmniAnomaly as their main comparator. Generative autoencoder baselines such as DAGMM appear mainly within the autoencoder family and have a minor presence elsewhere, which suggests that they are treated as specialized rather than universal yardsticks.

Several surveyed works compare their GNN-based models simultaneously against OmniAnomaly and other non-graph baselines such as LSTM based predictors, isolation forest, one-class svm, or autoencoder models including USAD and DAGMM. Across these comparisons, graph models usually outperform the non-graph baselines on at least one dataset, but the margin and stability of the gains vary. In benchmarks where the graph is constructed heuristically from tabular features and the induced relationships are weak or noisy, sequential and reconstruction based methods often remain competitive and sometimes match or exceed the GNN-based alternatives. When the domain provides a clear relational structure, for example physical topologies in industrial control systems, known dependencies between sensors, or explicit communication links between hosts, graph models tend to show more systematic improvements, especially in localizing which entities are anomalous and how anomalies propagate. Taken together, the evidence suggests that graph structure

does add value over simpler methods in some settings, but the advantage is not uniform across all datasets and tasks.

Table 11 also highlights the role of other widely used baselines. Static graph encoders such as GAT, GraphSAGE, and GCN appear less often than temporal graph models, partly because they were originally designed for snapshot classification and do not model dynamics directly. Methods such as MTAD GAT and GDN therefore dominate the dynamic graph column, while OmniAnomaly remains the main comparator in studies where graph topology is treated as secondary. The remaining traditional baselines, including USAD, DAGMM, MAD GAN, and TranAD, cover autoencoder, generative, and transformer families with lower citation counts. Their presence ensures that new proposals are compared not only against a single reference, but also against models with different inductive biases and training objectives.

5.4.2 Datasets

In this part, we examine the datasets most frequently used in the surveyed literature and how their properties affect the reported performance. Most of these datasets are multivariate time series or flow level tabular logs, where each row aggregates measurements or connections over a given time span. Only a few datasets are graph native, with nodes and edges explicitly provided. As a consequence, many GNN-based approaches start from tabular records and construct graphs afterwards by defining entities such as IP addresses, sensors, or services as nodes and inferring edges from communication patterns or correlations.

Below, we briefly describe datasets frequently appearing in the reviewed articles, highlighting their characteristics and, when relevant, their cybersecurity orientation:

- SWaT and WADI. SWaT is collected from a scaled-down industrial water treatment plant running 11 days continuously, with the first 7 days normal and the remaining 4 days under attack. WADI is from an extended testbed with more sensors/actuators, containing 16 days of operation (14 normal, 2 under attack).
- UNSW-NB15. Contains benign and 9 classes of malicious flows (Fuzzers, Analysis, Backdoors, etc.), representing realistic modern traffic (hybrid of real normal data and synthetic attack behaviors).
- BOT-IoT. Developed in a testbed using virtual machines, firewalls, and Node-RED. It contains mostly malicious IoT traffic (multiple subsets in different file formats and sizes). A common subset used is 5% of the overall dataset.
- NSL-KDD. A refined version of KDD Cup 99, removing redundant records to mitigate biases. It still focuses on network intrusion detection with labeled connection records.
- HAI. A cyber-physical system testbed simulating complex industrial processes, generating sophisticated attacks under realistic ICS conditions.
- Mirai. Focuses on IoT-based botnet activity. The dataset includes classes of normal traffic, ACKFlooding, SYNFlooding, and HTTPFlooding attacks, enabling analysis of multiple flooding patterns.
- ToN-IoT. Generated from IoT/IIoT sensors running various attacks such as cross-site scripting (XSS), password cracking, and MITM (man-in-the-middle).
- CIRA-CIC-DoHBrw-2020 (DOHBRW2020). Captures benign and malicious DNS-over-HTTPS (DoH) traffic. Includes classification layers for DoH vs. non-DoH and be-

Table 12 Reference table for the main datasets

Ref	Dataset	Year	Times used	Features	Anomalies	Total	Cyber-security oriented
Zhao et al. (2020)	SWAT	2016	28%	51	12.14%	925,119	◖
Wu et al. (2020)	SMAP	2018	26%	25	13.13%	562,800	◯
Hundman et al. (2018)	MSL	2018	25%	55	10.72%	132,046	◯
Ahmed et al. (2017)	WADI	2017	23%	127	5.99%	1,221,372	◖
Su et al. (2019)	SMD	2019	16%	38	4.16%	1,416,825	◖
Abdulaal et al. (2021)	PSM	2021	9%	25	27.76%	220,322	◖
Moustafa and Slay (2015)	UNSW-NB15	2015	4%	49	12.65%	2,540,044	●
Koroniotis et al. (2018)	BOT-IOT	2018	4%	46	–	73,360,900	●
Wibowo et al. (2019)	NSL-KDD	2009	3%	42	51.7%	148,517	●
Kunegis (2013)	UCI	2009	4%	–	–	59,835	◯
Shin et al. (2020)	HAI	2020	1%	84	2.2%	921,603	◯
Kalupahana Liyanage (2020)	MIRAI	2020	1%	47	52.47%	201,061	●
Moustafa (2021)	TON-IOT	2019	1%	42	96.37%	21,978,632	●
Montazerishatoori et al. (2020)	DOHBRW2020	2020	1%	29	92.65%	269,643	●
Kang et al. (2019)	IOT NETWORK INTRUSION	2019	1%	42	41.18%	2,985,994	●
Stolfo et al. (1999)	KDD CUP 1999	1999	1%	41	19.69%	4,898,431	●
Sharafaldin et al. (2018)	CIC-IDS-2017	2017	1%	79	25.93%	2,830,743	●
Sarhan et al. (2022)	NF-TON-IOT-V2	2022	1%	43	96.44%	22,339,021	●
Sarhan et al. (2022)	NF-BOT-IOT-V2	2018	1%	42	99.64%	37,763,497	●

We use a filled circle (●) for datasets clearly oriented toward cybersecurity, a half-filled circle (◖) for those covering cybersecurity only partially (e.g., critical infrastructure, IoT networks, or SCADA systems), and an empty circle (◯) for datasets with no explicit cybersecurity orientation

nign vs. malicious DoH.

- IoT Network Intrusion. Includes denial-of-service, UDP floods, port scanning, and other network-based attacks observed in real IoT traffic captures.
- KDD Cup 1999. A classic benchmark for network intrusion detection, consisting of simulated intrusions in a military network environment. Predecessor to NSL-KDD.
- CIC-IDS-2017. Contains benign and modern common attacks, with packet captures and labeled flows (timestamps, IPs, ports, and protocols).
- NF-TON-IoT-V2 / NF-BOT-IoT-V2. NetFlow-based versions of the ToN-IoT and Bot-IoT datasets with a high percentage of malicious traffic (over 96% for NF-TON-IoT-V2 and over 99% for NF-BOT-IoT-V2).
- Social Bot Datasets (BOTWIKI-2019, PRONBOTS-2019, CRESCI-2017, CRESCI-2015). These revolve around detecting social-media bots on X (formerly Twitter)

and include manually verified benign accounts alongside various categories of spambots and fake followers.

Table 12 complements this list with aggregated statistics such as size, dimensionality, anomaly ratio, and degree of cybersecurity orientation. Empirical work gravitates toward a small group of relatively compact benchmarks. SWAT (51 features, 0.9 million rows) dominates and is often analyzed together with SMAP, MSL, or WADI; similarly, studies that include SMD frequently report companion results on PSM. These datasets were collected on controlled testbeds or curated industrial pilots, with clearly delimited attack periods and a limited variety of operating conditions. This design reduces noise and makes model training easier, but it also means that the learning problem is substantially simpler than anomaly detection on raw traffic in large operational networks. High detection rates are therefore likely to overestimate the performance that can be sustained in more heterogeneous and weakly supervised environments.

The ◖ markers do not denote a lack of security relevance; rather, they flag datasets whose threats target cyberphysical processes instead of conventional networks. SWAT and WADI, for example, encompass false data injection, denial of service, and sensor spoofing incidents that would be impossible to replicate in open Internet traces.

In recent years, several traffic collections with more than twenty million records, such as ToN-IoT, NF-TON-IoT-V2, and NF-BOT-IoT-V2, have appeared. They reflect a growing interest in attacks against connected devices and the need for realistic, high volume baselines. However, these larger corpora are still used much less frequently than the smaller benchmarks listed above.

Most datasets are heavily skewed toward the normal class. The balanced exceptions, NSL-KDD and MIRAI, achieve parity by design: the former was curated to remove the redundancy and class bias of KDD Cup 1999 for competition purposes, while the latter was generated in laboratory to capture botnet flows under controlled conditions. In real deployments, the anomaly rate is usually much lower, and labels are incomplete or noisy. Reported metrics on balanced or mildly imbalanced corpora should therefore be read as optimistic upper bounds rather than realistic estimates of performance over long periods.

A further source of bias comes from the way graphs are built from tabular data. When nodes and edges are defined through simple heuristics, for example by linking entities that co-appears in short time windows or share similar feature values, the resulting graphs can be easier to model than raw traffic and may reflect specific design choices of the authors. On small and carefully curated datasets this can amplify apparent performance gains, since the constructed structure often highlights the same patterns that guided the labeling process. These conditions are useful for exploring new ideas, but they reduce the extent to which reported metrics can be interpreted as direct evidence of how GNN-based detectors will behave on large, noisy, and partially observed operational networks.

Table 13 Frequencies of commonly used metrics in the reviewed studies

Metric	Frequency
F1-Score	67%
Precision	54%
Recall	52%
ACC	18%
AUC	15%
ROC AUC	6%

Table 14 Summary of reviewed GNN-based anomaly detection approaches

References	GNN library/framework	Architectures implemented	Temporal approach
King and Huang (2023) Euler	PyTorch	GCN	Time-ordered edges
Zhu et al. (2021) BotSpot++	PyTorch	Custom GCN+GAT	Window-based
Peng et al. (2024) UnDBot	PyTorch	Hybrid	Constructed from temporal data
Guo et al. (2024) Rustgraph	PyTorch	Autoencoder-based	Evolving snapshots
Li et al. (2022) ProGraph	No standard GNN library	Custom GCN+GAT	Constructed from temporal data
Sun et al. (2024) DeepHunt	PyTorch	GCN + Autoencoder-based	Sliding time window
Mendoza et al. (2020) bot detection	PyTorchBigGraph	Hybrid GNN	Cascades using timestamps
Zhao and Fink (2024) DyEdgeGAT	PyTorch	GAT	Evolving snapshots
Aslan and Choi (2023) VisGIN	PyTorch	Custom GCN	Time-ordered edges
Shi et al. (2023) TCN+VAE Hybrid	PyTorch	GCN	Time-ordered edges
Reddy et al. (2024) Cryptojacking detection	PyTorch	GCN	Evolving snapshots
Fan et al. (2023) ATF-UAD	DGL	GCN	Time-ordered edges

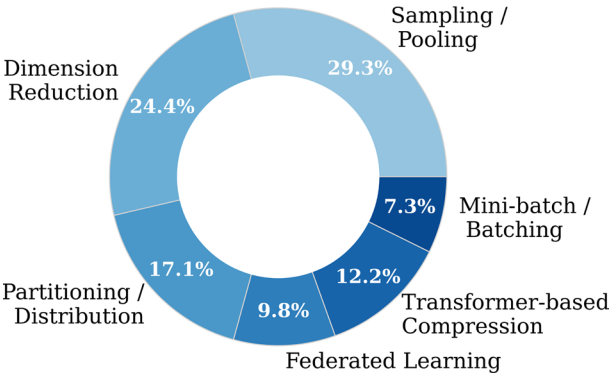

Fig. 18 Relative frequency of major scalability strategies. Scalability is most often addressed through a combination of graph reduction and distributed processing strategies

5.4.3 Evaluation metrics and biases

We examine now how the previously described metrics (see Sect. 2.6) have been employed in our surveyed literature and discuss their suitability for commonly used datasets in cybersecurity.

A closer look at the datasets analyzed in these studies reveals that many of them exhibit severe class imbalance, often reflecting real-world network traffic conditions where malicious events are relatively rare. Under such imbalance, Accuracy becomes less informative because it can be artificially inflated by predicting the dominant class. Consequently, the strong prevalence of F1-Score (67%) alongside Precision (54%) and Recall (52%) is justified, as these metrics are more sensitive to minority classes and better capture the trade-off between false alarms and missed detections. Some articles further employ ROC AUC or

generic AUC (15%), although these metrics can hide performance differences in heavily skewed conditions and should be interpreted with caution. Meanwhile, a modest subset of works still reports Accuracy (18%), likely due to its simplicity and historical use in intrusion detection benchmarks.

Considering the characteristics of widely adopted datasets such as BOT-IoT, UNSW-NB15, and NSL-KDD, the frequent reliance on Precision, Recall, and F1-Score appears more appropriate than raw Accuracy or ROC curves. BOT-IoT, for instance, can surpass 90% malicious traffic, making a high Accuracy meaningless if true negatives dominate. Similarly, UNSW-NB15 and NSL-KDD each contain multiple attack types, reinforcing the value of metrics like F1 or Macro-F1 in evaluating all classes fairly. In contrast, more balanced datasets may accommodate broader metric selections without risking misleading outcomes.

In addition to binary or multi-class classification tasks, a small fraction of papers employ regression-based metrics for forecasting phenomena such as the time-to-compromise or resource usage. Although RMSE and MAE can indicate average prediction errors, they provide minimal insight into whether early detection or severe outliers carry disproportionate importance. As a result, their utility should be weighed against domain-specific requirements and potential distribution skew.

Table 13 illustrates the relative frequency of each metric, these observations suggest that F1-Score, Precision, and Recall remain the most coherent choices for the highly imbalanced datasets prevalent in cybersecurity. Metrics like MCC can add further clarity, yet they are underutilized, possibly due to interpretability concerns. Simple measures such as Accuracy or ROC AUC still appear, but caution is advised when minority class detection is crucial. In practice, selecting an evaluation metric that aligns with the dataset's imbalance and the real-world implications of missed attacks versus false alarms is paramount for credible results.

5.5 RQ6: Reproducibility, deployment, and open challenges

This subsection synthesizes the practical obstacles and forward-looking proposals reported by the primary studies, focusing on reproducibility artefacts, evidence of real-world uptake, and the most recurrent engineering and data constraints. We also contextualize these barriers by the application domains that most often motivate graph-based systems.

5.5.1 Code availability

The availability of source code for reviewed GNN-based anomaly detection approaches varies widely. As summarized in Table 14, only a small minority of the surveyed GNN-based anomaly detection approaches (around 15%) provide a publicly accessible implementation, usually through a GitHub repository. The table lists twelve methods for which we were able to locate executable source code. Five further studies announce a future public release without specifying a stable repository or documentation (Yan et al. 2023; Wang et al. 2023; Qin et al. 2023; Liu et al. 2021; Li et al. 2022), and six works state that code may be shared upon request rather than being openly available (Zhang et al. 2023; Liao et al. 2024; Guan et al. 2024; Wen et al. 2024; Song et al. 2023). For most published results, independent readers therefore have no direct access to the underlying implementation.

When code is released, implementations are typically built on PyTorch, attributed to its dynamic computational graph paradigm and the extensive ecosystem of tools and pre-trained models, which facilitate rapid prototyping and ease of integration, with a few examples relying on DGL, built atop frameworks to offer abstractions and sparse graph operations, or PyTorchBigGraph tailored for embedding extremely large graphs. Architectures are mostly GCN-style models or simple hybrids, so the software stack is relatively uniform and, in principle, reusable across studies. The temporal dimension, in contrast, is handled in several different ways even within this small subset. Given the low number of open implementations and this heterogeneity, it is difficult to draw strong conclusions about which temporal strategy is preferable; the main observation is that temporal modeling choices remain diverse and only loosely standardised.

From a reproducibility perspective, the low code-availability rate has concrete consequences. Many papers provide only high-level descriptions of graph construction, temporal batching, and feature engineering. Without code, re-implementations must infer how flows are mapped to nodes, how edges are sampled over time, or how rare events are handled. Hyperparameters, random seeds, and early-stopping criteria are often only partially reported. Small differences in these choices can produce noticeable changes in anomaly-detection performance, so reproducing reported scores becomes uncertain. Several factors may explain why implementations are not shared more often, including non-disclosure agreements, security policies around operational network data, or the absence of strong artefact-evaluation requirements in many venues. Whatever the reason in each individual case, the end result is that many GNN-based anomaly detection methods are difficult to validate, compare, and adapt to new environments.

5.5.2 Real-world case studies vs. proofs of concept

Now we distinguish between studies that report some degree of deployment in operational environments and those that remain at the proof-of-concept stage. Only a few works describe integration into industrial platforms or extensive testbeds. Examples include BotSpot++ Zhu et al. (2021), deployed in the mobile advertising platform of Mobvista with online and offline validation; the adversarial GNN for time series of Zheng et al. (2024), integrated into GaussDB (DWS) by Huawei; the holistic IoT security platform from the SerIoT project Protogerou et al. (2022), tested on intelligent transport systems with on-board units (OBUs) and roadside units (RSUs); and DeepHunt Sun et al. (2024), developed with industrial partners such as Huawei and Alibaba, although deployment at scale is not documented.

Most other papers are evaluated on offline datasets, synthetic environments, or small laboratory setups. These studies are useful to explore new architectures and to compare methods under controlled conditions, but they offer limited evidence about long term operation, resilience to changing traffic, or interaction with existing security workflows. From the perspective of adoption, the very small number of documented deployments in real environments suggests that GNN-based anomaly detection is still at an early stage of industrial uptake.

The situation in practice is likely more nuanced. Academic models are rarely transferred to production as complete systems that can be used directly; instead, ideas, architectures, and graph construction strategies are often re-implemented inside broader proprietary platforms. Such integrations are not always described in the scientific literature, may be cov-

ered by non-disclosure agreements, and frequently leave no trace in public indexes. Even if actual industrial use is higher than the published record suggests, the lack of detailed public case studies means that the community has little shared evidence on how these methods behave in production, which design choices are sustainable over time, or which failure modes appear once they are exposed to real traffic.

5.5.3 Limitations and future directions

In this subsection, we analyze the primary limitations identified across the surveyed articles, i.e. the obstacles related to computational complexity, data availability, and interpretability challenges. Many works offer partial or comprehensive cybersecurity insights, but still encounter issues such as insufficient real-world datasets, high computational overhead, and limited explainability for security analysts.

Scalability and real-time constraints

Controlling computational complexity in massive, fast-evolving graphs remains a primary hurdle. Many studies highlight the difficulty of handling large-scale or dynamic topologies under strict latency constraints (L(y)u et al. 2023; King and Huang 2023; Yan et al. 2023; Fu et al. 2024; Akram et al. 2024; Wang et al. 2023; Liu and Liu 2023; Protogerou et al. 2022). In practice, the cost grows with the number of edges that must be processed at each update and with how often the graph is re-encoded across time (e.g., snapshot sequences or sliding windows), making naive temporal pipelines hard to deploy at scale. Empirical reports reflect this pressure: Zhu et al. (2021) describe training times on large network data in the 1–7 h range, while King and Huang (2023) reduce time and memory usage by adjusting snapshot granularity and parallel workers. In collaborative settings, some studies note that federated training can increase the overall cost by roughly $1.5\times$ to $3\times$ relative to centralized learning Jiang et al. (2022).

Across the surveyed works, scalability is typically pursued by combining a small set of recurring strategies. Partitioning/distribution, splitting computation across machines or processing snapshots in parallel; sampling/pooling, top-k selection, neighborhood sampling, or subgraph extraction; federated/distributed learning, training without centralizing data; dimension reduction, compressing high-dimensional telemetry; transformer- or wavelet-based compression, summarizing long-range context without processing all tokens at full resolution; and mini-batching, controlling memory footprint during training. These choices reflect a shared trade-off: methods that preserve more temporal detail tend to require stronger reductions or distribution to remain feasible, whereas lighter models may keep more of the raw structure but sacrifice long-range context. For example, several works prune or sample the effective graph to bound per-step computation L(y)u et al. (2023); Zhu et al. (2021), while others distribute training across sites or devices when data cannot be centralized Jiang et al. (2022); Zhang et al. (2023); Protogerou et al. (2022), accepting additional communication overhead.

Figure 18 summarizes how frequently each strategy is explicitly reported as a primary scalability mechanism in the reviewed articles (mention frequency reflects reporting emphasis rather than a direct measure of operational impact). In this context, sampling/pooling covers top-k selection and subgraph extraction Liu et al. (2021); Aslan and Choi (2023), partitioning/distribution refers to splitting computation across machines or parallel snapshot processing Yan et al. (2023); Peng et al. (2024); Presekal et al. (2023), and Transformer- or

wavelet-based compression reduces temporal overhead by summarizing long-range dependencies Wang et al. (2023); Liu et al. (2021); Wang et al. (2023); Wen et al. (2024); Xie et al. (2024). Because each technique targets a different bottleneck (graph size, hardware limits, or temporal context), most scalable pipelines combine at least two of them to achieve acceptable latency without collapsing the anomaly signal.

Limited real-world datasets or domain mismatch

Real-world data—especially in industrial, IoT, or cybersecurity scenarios—tend to be incomplete or noisy, with few ground-truth labels available (He et al. 2024; Zhang et al. 2024; Sun et al. 2024; Zhou et al. 2022; Pang et al. 2024). Designing robust GNN-based anomaly detection methods under partial or uncertain supervision remains an open challenge. In many cases, privacy regulations and proprietary constraints further restrict access to labeled data, making it difficult to establish widely accepted benchmarks. Some works partially address this gap through federated learning (Jiang et al. 2022) or synthetic data generation pipelines, yet these solutions do not guarantee coverage of all real-world anomalies.

Data acquisition and quality also play a decisive role in whether models generalize. A number of articles rely on simulated or domain-specific datasets (He et al. 2024; Presekal et al. 2023; Alrumaih and Alenazi 2024), limiting their applicability across different verticals. Meanwhile, cybersecurity research often depends on specialized intrusion detection benchmarks (e.g., NSL-KDD, UNSW-NB15, see Sect. 5.4.2) that may not reflect modern threats or operational constraints. Some authors attempt domain adaptation (He et al. 2024) or transfer learning, but these methods are still relatively immature for GNN, particularly when the network structure and threat landscape differ significantly across domains. Consequently, performance on one dataset might not translate to another, creating potential mismatches when deployed in real environments.

As discussed in Sect. 5.4.2, the majority of publicly available cybersecurity datasets are originally tabular, with limited inherent graph structure. Researchers must manually construct node and edge relationships–defining nodes as IP addresses or hosts, and edges based on observed protocols or traffic flows. While such conversions enable GNN-based analyzes, they risk oversimplifying network interactions and ignoring context-specific nuances. Thus, bridging the gap between tabular or simulated data and complex real-world graphs remains a pressing concern, underscoring the need for more extensive, standardized benchmarks or collaborative data-sharing efforts that capture genuine diversity in network topologies and threat vectors.

Hyperparameter dependencies and high dimensionality

Many GNN-based solutions point out the difficulty of modeling intricate spatial-temporal relationships in high-dimensional settings, particularly when dealing with multivariate time series or detailed sensor logs (He et al. 2024; Qin et al. 2023; Yang and Yue 2023; Liu et al. 2021; Zhang et al. 2023; Wang et al. 2023; Li et al. 2022). Capturing subtle correlations across numerous nodes and features can lead to prohibitive computational overhead and increase the risk of overfitting, especially if the system must process large volumes of data in near-real-time. Techniques like hierarchical graph pooling, feature selection, or dimension reduction can mitigate some of these issues, though no consensus has emerged on the best practice for balancing expressiveness and efficiency.

An additional concern is the strong reliance on hyperparameter tuning and manual thresholds. GNN performance in high-dimensional contexts often hinges on selecting appropriate layer sizes, sampling rates, or regularization parameters. Some authors suggest auto-tuning

strategies (e.g., Bayesian optimization) to streamline this process (Zhang et al. 2024; Fan et al. 2024, 2023), while others discuss self-adaptive systems capable of dynamically adjusting hyperparameters based on the evolving data distribution. Overall, automating hyperparameter selection and reducing the reliance on fixed thresholds remain open research directions, particularly in production-grade anomaly detection tasks where data patterns can shift abruptly.

Interpretability and explainability Real-world adoption of GNN-based anomaly detection often depends on whether alerts can be justified to security analysts or domain experts (Peng et al. 2024; Sun et al. 2024; Yang and Yue 2023; Benzaïd et al. 2023; Liu et al. 2023). Despite promising detection results, most surveyed studies provide limited transparency about why specific nodes, edges, or time segments are flagged. Some papers include lightweight visual aids such as heatmaps or node importance rankings (Benzaïd et al. 2023; Song et al. 2023), while others report localization of anomalous entities with little supporting evidence beyond the final score (Liao et al. 2024; Chen et al. 2023). A smaller subset side steps the full nonlinear GNN pipeline by restricting the model class to retain interpretability (Li et al. 2022). Overall, explanations are usually presented as qualitative add ons rather than as outputs evaluated with the same rigour as detection performance.

Several works acknowledge the need for transparency in safety and security critical domains (He et al. 2024; Sun et al. 2024). Mechanisms such as attention modules (Zhao and Fink 2024; Gao et al. 2023), structural entropy scores (Peng et al. 2024), or probabilistic and Variational Autoencoder (VAE) based components (Wang et al. 2023; Mir et al. 2024; Shi et al. 2023) can provide partial insight into which features or connections influence anomaly scores. However, these signals are rarely validated as explanations. In particular, few studies assess whether the highlighted nodes or edges are stable across runs, consistent across time, or aligned with domain knowledge, and almost none compare explanation quality across competing models.

This leaves a clear methodological gap, interpretability is discussed, but it is not benchmarked. Established post-hoc explainers, such as GNNExplainer(Ying et al. 2019), PGExplainer(Luo et al. 2020), or GraphSHAP(Perotti et al. 2023), are largely absent from the literature. This omission matters because these methods offer a principled way to attribute predictions to compact subgraphs and feature subsets, enabling comparisons under shared criteria. Their limited uptake suggests that many papers rely on bespoke visualizations instead of using available explainability tools that would make results easier to audit and reproduce.

A practical way forward is to treat explanations as first class evaluation targets. Alongside detection metrics, future work could report explanation fidelity (whether the explanation preserves the model output), sparsity, and stability across random seeds and adjacent time windows. Reporting the explanation protocol and settings would also improve reproducibility, and it would help analysts use explanations for triage and root cause analysis in operational settings.

Difficulty in generalizing solutions

Environments in cybersecurity often evolve rapidly, introducing new devices, changing traffic patterns, or upgraded infrastructures (e.g., industrial control systems (Alrumaih and Alenazi 2024), IoT deployments (Liu and Liu 2023), and enterprise networks). Such diversity demands GNN frameworks capable of adapting to novel conditions without sacrificing detection performance (He et al. 2024; Zhang et al. 2024; Presekal et al. 2023; Zhao and

Fink 2024). Unfortunately, domain-specific tuning or training can limit a model's applicability across heterogeneous data sources, leaving domain generalization as an ongoing obstacle.

To address these discrepancies, some authors propose modular designs tailored to particular environments. For instance, integrating specialized modules for ICS Alrumaih and Alenazi (2024), or focusing on short-lived events in IoT-based traffic flows Liu and Liu (2023). Although these approaches often excel in their respective settings, they can struggle when transferred to unrelated domains. Developing universal or domain-agnostic GNN solutions requires flexible architectures capable of managing structural shifts, different feature distributions, and varying label definitions–an ambitious goal that remains largely unmet in current literature.

Moreover, high variability within a single domain (for example, a rapidly growing IoT network) can introduce new device types, protocols, or threat vectors unseen at training time. Continuous learning, transfer learning, or meta-learning strategies may help a GNN adjust to such changes incrementally, but implementations tailored to dynamic anomaly detection remain relatively rare. Until these methods mature, practical deployment may still hinge on frequent retraining or manual fine-tuning whenever network conditions drift beyond the scope of the original dataset or model assumptions.

Privacy and security for collaborative settings Large-scale cybersecurity deployments often span multiple organizations or geographically distributed sensors. In such scenarios, data-sharing restrictions, regulatory constraints, and privacy concerns can impede direct access to raw logs or feature sets. Federated learning Jiang et al. (2022); Zhang et al. (2023) and other distributed approaches offer solutions by training GNN-based anomaly detectors without centralizing sensitive data. However, these methods face notable hurdles. One is the increased computational and communication overhead: as previously discussed, observing a 1.5 to 3 cost increase compared to non-federated training Jiang et al. (2022). Another is ensuring trustworthy model aggregation, where compromised nodes or adversarial participants might inject malicious gradients, skew local training data, or sabotage global updates.

Hence, protecting data confidentiality while preserving robust detection performance is an ongoing challenge. Some works propose cryptographic tools or secure multi-party computation for aggregating model parameters, but these methods can further inflate training times and complicate system design. Similarly, techniques for detecting malicious updates (e.g., gradient-checking heuristics or anomaly detection among federated nodes) remain largely experimental. Adapting these safeguards to dynamic graphs adds another layer of complexity, as the graph structure itself can shift over time, altering which nodes contribute to each training iteration.

Even beyond malicious intent, domain mismatch and varying local data distributions pose serious risks for federated or distributed GNN. A model trained under normal operating conditions at one organization might encounter dramatically different threat vectors at another, thus limiting generalization. Robust solutions may require ongoing exchange of refined embeddings or subgraph statistics, combined with adaptive weighting schemes that adjust to local anomalies in near real time. Although efforts to combine federated learning with dynamic GNN have shown promise, scaling these approaches to wide-scale production environments calls for additional research into fault tolerance, secure parameter sharing, and reliable detection of poisoned updates.

Table 15 Recurring limitation themes and typical mitigation directions

#	Limitation	Future proposals/example articles	Security focus	Scalability	Interpretability	Domain flexibility
1	High computational cost and scalability issues (e.g., training time and memory)	Parallelization and graph partitioning; future work involves GPU/TPU acceleration King and Huang (2023); Altaf et al. (2024); Fu et al. (2024); Miao et al. (2023)	◐	●	○	●
2	Limited real-world datasets or domain mismatch	Domain adaptation frameworks; emphasis on synthetic-to-real transfer learning He et al. (2024); Presekal et al. (2023)	○	●	○	○
3	Lack of thorough interpretability/explainability	Attention mechanisms, local explanation methods, explainable AI toolkits L(y)u et al. (2023)	●	○	●	●
4	Strong dependence on hyperparameter tuning and manual thresholds	Auto-tuning, Bayesian optimization; future directions include self-adaptive systems Zhang et al. (2024); Fan et al. (2024, 2023)	○	●	●	●
5	Sensitivity to noisy or incomplete graph data	Robust anomaly detection strategies; future work: outlier-resistant graph encoders Zheng et al. (2024); Wang et al. (2023); Mir et al. (2024)	○	●	●	●
6	Difficulty in generalizing solutions to diverse cyber domains (ICS, IoT, enterprise networks)	Modular designs tailored to ICS and IoT; next steps: domain-agnostic frameworks Alrumaih and Alenazi (2024); Liu and Liu (2023)	●	●	○	●

○ (none/low), ◐ (partial/medium), ● (high)

Most limitations primarily affect scalability and cross-domain flexibility, while interpretability concerns emerge as a distinct barrier for analyst-facing adoption

To contextualize how limitations are emphasized in prior work, Fig. 19 summarizes the frequency of the major limitation categories we distilled from the literature. The Computational cost & scalability category clearly dominates, followed by Limited real-world datasets / domain mismatch and Difficulty generalizing solutions to diverse cyber domains. A relatively small fraction of articles explicitly mention real-time or streaming constraints as a key limitation, suggesting a gap in the research on deploying dynamic GNN in live enterprise or industrial environments.

A consolidated overview of the most frequently reported limitation categories, highlighting their relevance for cybersecurity applications, appears in Table 15. Notably, high computational cost and insufficient real-world datasets often emerge as dominant concerns, while additional issues such as interpretability, hyperparameter tuning, noisy/incomplete data, and domain mismatch underscore the need for more robust, generalized solutions. Ensuring that these concerns are systematically addressed remains pivotal for bridging the gap between theoretical advances and operational deployments of GNN-based anomaly detection in dynamic cyber domains.

5.5.4 Domains motivating graph-based systems

This part situates the reviewed methods in the operational domains where they are most frequently evaluated, complementing the discussion of RQ6. The intent is not to compare performance across sectors, but to show which application contexts repeatedly motivate dynamic or temporal graph constructions in the surveyed studies.

Figure 20 summarizes this distribution. The percentages reflect the fraction of references that mention a given domain (categories are not mutually exclusive, as a single paper may cover multiple settings). Communications-related scenarios dominate, followed by IoT and industrial environments, which is consistent with the prevalence of highly connected systems where anomalies manifest themselves as changes in interaction patterns rather than isolated feature deviations.

The first group concerns communications and networking infrastructures (e.g., 5 G/B5G, edge/cloud-managed networks), where graph representations naturally encode dependencies among devices, services, and routes. In these settings, anomalies are often tied to evolving connectivity or traffic relations, which aligns well with graph-based modeling.

A second group covers IoT, industry, and critical infrastructure, where cyber incidents and faults are tightly coupled to physical processes. Graphs capture sensor dependencies, device-to-device interactions, and subsystem coupling, enabling localization at the level of affected components and their contributing relations.

A third group includes mobility and transport, energy and environment, and digital platforms. Here, graph learning is used to model coordinated behavior (e.g., fleets and road interactions, grid dependencies, content propagation), and anomalies frequently correspond to coordinated patterns or abrupt shifts in relational structure. These domains reinforce that the same modeling abstractions used in cybersecurity environments generalize to other rich interaction systems, provided that the graph construction and evaluation protocol are made explicit.

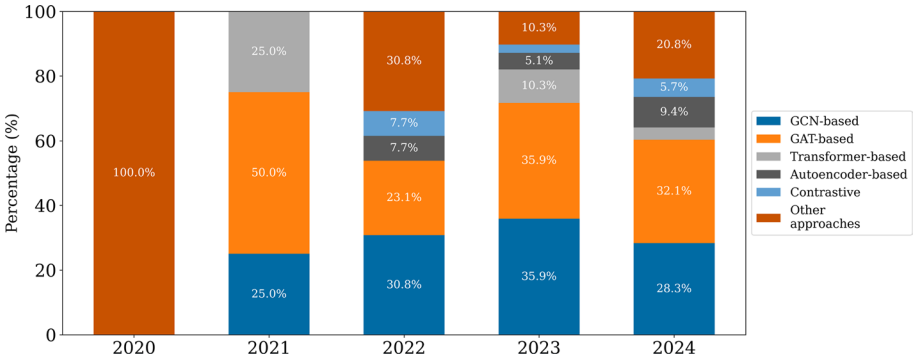

Fig. 14 Year-by-year distribution of architectural families, showing increasing architectural diversity in recent years while GCN and GAT remain recurrent baselines

6 Discussion

This section revisits each research question from a synthesis perspective. We distill the main points of agreement across studies (what is reasonably established under the reported protocols) and highlight the most consistent gaps that remain unresolved.

6.1 Discussion on RQ1

The reviewed literature shows a clear acceleration in publication volume toward last years (Fig. 10), together with a gradual broadening of methodological choices rather than convergence to a single dominant recipe. Over the same period, unsupervised training becomes more prominent (Fig. 12), GCN/GAT remain the most recurrent backbones while alternative families appear more often (Fig. 14), and prediction targets shift from mostly node-centric outputs toward a larger share of joint node–edge formulations (Fig. 16), which is consistent with an increasing focus on evolving dependencies and interaction-driven anomalies. What remains open is whether these trends reflect durable maturation or short-term dataset and benchmark effects. The evidence is still concentrated on a small set of commonly reused evaluation protocols, and many studies do not report enough detail on graph construction, temporal splitting, or deployment constraints to assess how robust the observed shifts are across operational settings.

6.2 Discussion on RQ2

Across the surveyed works, supervision choices consistently reflect label availability rather than a universal performance ranking. Unsupervised objectives are most prevalent and are typically framed as deviation scoring that avoids dependence on a stable attack taxonomy, which makes them practical when labels are incomplete, noisy, or quickly outdated. When labels exist, supervised pipelines are often benchmark-driven and tend to report stronger point performance, but many studies also surface within-study sensitivity to label quality and coverage (e.g., sparse labels, noise, or temporal aging) under the same dataset and protocol. Semi-supervised designs explicitly explore this trade-off by varying the labeled

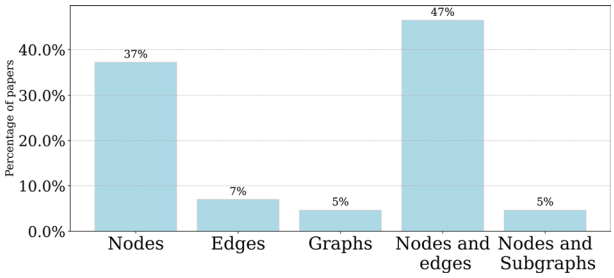

Fig. 15 Distribution of studies based on the analysis level. Node and joint node–edge prediction targets dominate the studies, whereas edge and graph-level objectives are comparatively rare

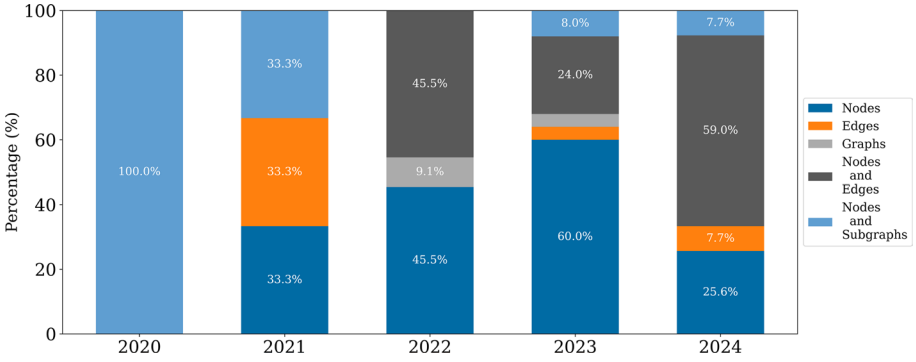

Fig. 16 Evolution of graph analysis. Over time, shifts from predominantly node targets toward more joint node–edge formulations, while edge and graph level objectives remain niche

fraction, while self-supervised setups are mostly justified through within-study comparisons against the same backbone without pretraining. In parallel, the inductive–transductive distinction remains a central deployment trade-off. Transductive settings can yield strong reported metrics when the full graph is available at training time, whereas inductive designs are better aligned with operational requirements in which new hosts, services, or interactions appear after deployment.

Two gaps limit how actionable these findings are for real deployments. First, the field still lacks consistent and standardized ways to test supervision choices under realistic label dynamics (e.g., controlled studies of label aging, concept drift, and label noise with shared protocols), which makes it difficult to compare the robustness of paradigms beyond individual papers. Second, inductive generalization is often stated as a requirement, but is not uniformly evaluated under explicit out-of-distribution conditions (new nodes, new subnets, new devices, or shifted traffic regimes). More systematic evaluation practices that jointly vary label availability and distribution shift within a fixed experimental setup would clarify when the extra complexity of semi-/self-supervision or inductive training translates into reliable gains for cybersecurity monitoring.

6.3 Discussion on RQ3

The reviewed corpus relies heavily on GCN- and GAT-based backbones, as summarized in Table 6, with both families serving as strong, low-engineering defaults that are widely avail-

able in mature libraries. Over time, Fig. 14 suggests a gradual diversification: GCN/GAT remain recurrent baselines, but a broader mix of alternatives appears in later years. Most works still adopt a single dominant backbone, and multi-family combinations are comparatively rare in Fig. 13, which is consistent with the higher engineering and tuning cost of hybrids and the fact that gains are not always clearly attributable when multiple strong blocks are blended. Concerning temporal integration, Table 7 indicates that many studies prefer dynamic formulations built from evolving snapshots, typically pairing lightweight message-passing with an explicit temporal component (recurrence, temporal convolutions, or attention across snapshots). Temporal-edge formulations are less common but place more emphasis on selectively weighting events, aligning with the higher share of attention-based encoders in that branch. Within-study comparisons reported in Table 9 repeatedly support the same qualitative conclusion, adding temporal memory or temporal retention tends to improve detection when threats unfold over multiple steps, and authors often justify this choice by benchmarking against their own static or short-horizon baselines.

Despite clear usage patterns, the literature still provides limited controlled evidence on when a change of backbone is materially beneficial once temporal modeling and training objectives are fixed. Many papers introduce temporal components together with other changes (graph construction rules, losses, sampling, windowing), which makes it hard to separate the contribution of the encoder family from the contribution of the temporal mechanism under a shared protocol. Hybrid architectures are another open area, Fig. 13 shows that they are used, but the cost–benefit boundary is not well characterized, especially when additional hyperparameters and scheduling complexity are introduced. Finally, temporal design is often evaluated on a narrow set of splits and horizons; more systematic stress tests that vary window length, event sparsity, drift intensity, and out-of-distribution entities would clarify which temporal encodings remain stable under realistic nonstationarity, and which optimizations in Table 9 generalize beyond the specific datasets used in each study.

6.4 Discussion on RQ4

Across the surveyed studies, prediction targets concentrate on node-level and joint node–edge objectives (Fig. 15 and Table 10), which aligns with how cybersecurity operators typi-

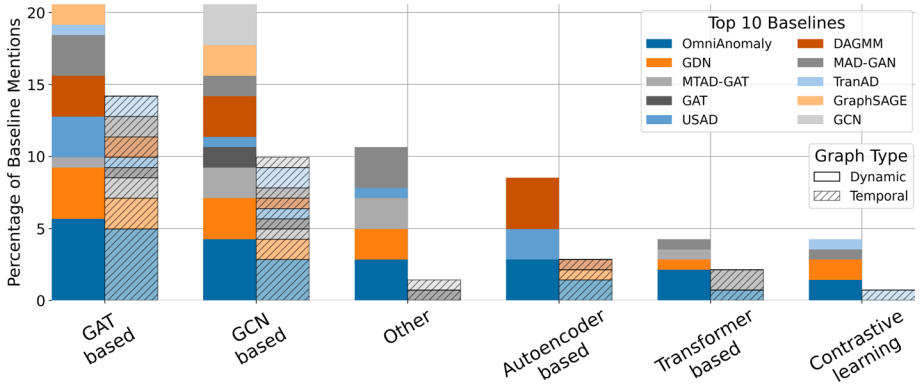

Fig. 17 Percentage of baseline mentions grouped by architectural family and graph type. Solid bars correspond to studies that explicitly model dynamic graphs, whereas hatched bars represent methods operating on temporal graphs. Each color slice marks one of the ten most-cited baselines

cally act on alerts. They need to identify affected entities and, increasingly, the interactions that justify the alarm. Edge-only objectives appear mainly when interactions are the natural unit of analysis (e.g., temporal links or flow relations), while graph-level prediction remains uncommon because many monitoring pipelines require fine-grained localization rather than a single label for an entire snapshot. The granularity rationale in Sect. 5.3.2 also indicates a recurring mapping between domain and representation, sensors/tags for ICS telemetry, hosts or IPs for traffic graphs, and accounts for social/bot settings, with temporal granularity controlled through event-level representations or windowed snapshots. Over time, Fig. 16 suggests a shift from predominantly node-centric outputs toward joint node–edge formulations, consistent with a growing emphasis on modeling evolving dependencies and with tooling that lowers the barrier to relational architectures.

Despite clear descriptive patterns, granularity is rarely treated as an experimental variable. Only a subset of works explicitly evaluates alternative spatial or temporal resolutions under the same protocol, so it is often unclear whether reported improvements stem from the GNN architecture or from representation choices such as node definitions, edge induction rules, or windowing strategies. This gap is particularly important because graph construction from tabular logs can encode strong assumptions that affect both detectability and interpretability, and because temporal aggregation can either stabilize detection or hide short-lived attacks. More systematic within-study ablations that vary (i) node/edge semantics, (ii) edge construction rules, and (iii) temporal resolution would clarify the accuracy–cost–explainability trade-offs implied by different granularities and help translate laboratory results into deployment-ready guidance.

6.5 Discussion on RQ5

Baseline selection is relatively concentrated. Table 11 and Fig. 17 show that a small set of reference methods (notably OmniAnomaly and GDN) anchors many comparative studies. These baselines play two distinct roles, OmniAnomaly tests whether adding relational inductive bias improves over a strong sequential detector without explicit topology, while GDN provides a lightweight temporal graph baseline that already combines graph and tem-

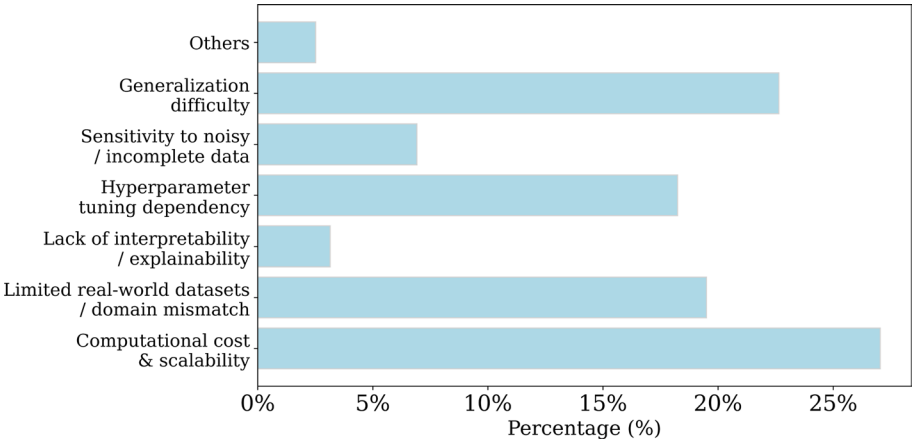

Fig. 19 Frequency of limitations. Computational cost and scalability dominate, while generalization and dataset realism/domain mismatch form the next most common concerns

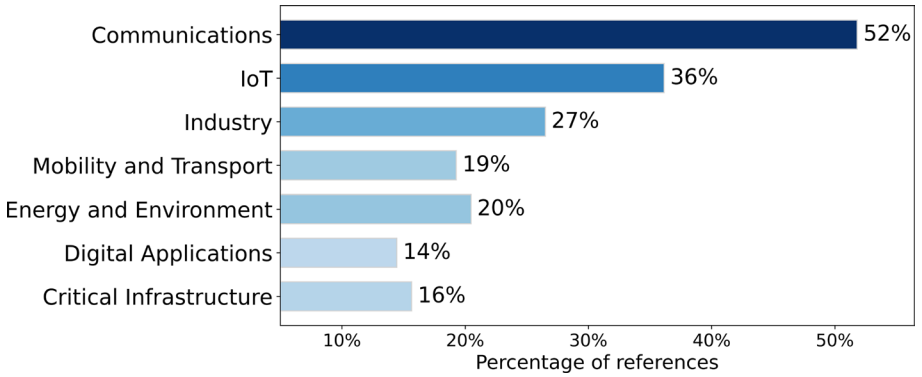

Fig. 20 Operational domains most frequently reported when motivating, validating, or benchmarking graph-based methods. Communications and IoT are the dominant application contexts

poral modeling. The pattern is therefore not only frequency-driven but also methodological. Many papers validate their contribution by positioning it between a strong non-graph baseline and a compact graph-based reference under the same protocol. Dataset usage is likewise concentrated. Section 5.4.2 and Table 12 indicate a heavy reliance on a small set of controlled benchmarks, with comparatively fewer studies using very large, high-volume traffic collections. A further recurring evaluation property is that many datasets are not graph-native, so performance depends jointly on the detector and on the graph construction heuristic derived from tabular records. Finally, metric reporting reflects the class-imbalance typical of security data. F1-score, precision, and recall dominate (Table 13), which is generally coherent when the minority class is the object of interest and false alarms versus missed detections must be balanced.

Several threats to validity recur across these evaluation choices. First, although baseline sets often include both graph and non-graph references, the evidence base is still fragmented because studies vary widely in graph construction, windowing, splits, and preprocessing, making cross-paper performance comparisons unreliable. Second, the prevalence of tabular-to-graph conversion introduces a representation bias, when edges are induced through heuristics that mirror labeling or simplify the underlying interactions, reported gains may reflect the constructed structure as much as the GNN itself, and ablations that isolate this effect remain uncommon. Third, metric usage is still inconsistent with deployment needs in some cases: accuracy and ROC-based measures can be misleading under severe skew, and the limited uptake of more informative summaries (e.g., macro-averaged measures or MCC) suggests that evaluation often prioritizes conventional reporting over robustness checks. Overall, stronger validation protocols that vary graph construction choices, label noise/sparsity, temporal splits, and imbalance levels, while reporting metrics aligned with operational costs, would reduce bias and make claims about generalization and practical utility more defensible.

6.6 Discussion on RQ6

Reproducibility is still limited by scarce artifact release. Table 14 shows that only a small subset of methods provide runnable public code, and even within that subset the handling of temporal structure (snapshots, windows, time-ordered events) remains heterogeneous, which

reduces direct usability. Published evidence of operational deployment is also sparse: only a few papers describe integration into industrial platforms or extensive testbeds, while most evaluations remain proof-of-concept studies on offline benchmarks or controlled environments. When authors discuss barriers explicitly, the same themes recur. Figure 19 indicates that computational cost and scalability constraints dominate reported limitations, followed by dataset realism/domain mismatch and difficulties in generalization across cyber domains; Table 15 links these themes to typical mitigation directions (e.g., partitioning/sampling, domain adaptation, and modular designs). The domain distribution in Fig. 20 further suggests where these constraints are most acute. Communications, IoT, and industrial/critical infrastructure settings repeatedly motivate dynamic graph modeling, which is consistent with environments where anomalies manifest as evolving dependencies and interactions rather than isolated feature deviations.

7 Conclusion

This systematic literature review shows that GNN-based anomaly detection has become an increasingly common approach in cybersecurity, especially when the data are naturally represented as dynamic or temporal graphs. By modeling entities and interactions as evolving relational structures, these models can capture dependencies and behavioral shifts that are hard to express with feature-only pipelines.

At the same time, the surveyed studies expose recurring barriers to deployment. Scalability is still a limiting factor in large, fast-changing networks where detection must run under tight latency and memory constraints; practical solutions based on partitioning, subgraph sampling, incremental updates, or federated learning are promising, but they are not yet consistently validated under production-like loads. The shortage of robust, real-world graph datasets also restricts generalization, motivating stronger domain adaptation protocols and more realistic synthetic data generation that preserves temporal and structural properties. Interpretability remains another bottleneck, many models provide limited rationale for flagged anomalies, which conflicts with analyst workflows that require actionable explanations and traceable evidence.

Future work should prioritize methods that align model design with operational constraints (streaming updates, concept drift, and resource budgets) while improving reproducibility and comparability across studies. Hybrid architectures that combine GNNs with sequence modeling, self-supervised objectives, and privacy-preserving training can help, but their value will ultimately depend on evaluation setups that reflect real security settings and on closer interaction with practitioners to define threat models, labeling policies, and success criteria that translate into deployable detectors.

Metrics

Table 16 shows the binary confusion matrix used to define the metrics reported in Table 17.

Table 16 Binary classification confusion matrix

	Predicted	
Actual	True positive (TP)	False negative (FN)
	False positive (FP)	True negative (TN)

Table 17 Metrics and equations for classification, regression, and efficiency tasks

Use	Metrics	Formula
Binary classification evaluation	Precision	$\frac{TP}{TP+FP}$
	Accuracy	$\frac{TP+TN}{TP+FP+FN+TN}$
	Recall	$\frac{TP}{TP+FN}$
	F1 score	$\frac{2\times\text{Precision}\times\text{Recall}}{\text{Precision}+\text{Recall}}$
	AUC	$TPR=\frac{TP}{TP+FN},\quad FPR=\frac{FP}{FP+TN}$
	Equal error rate (EER)	$\text{EER}=\text{FMR}=\text{FNMR}$
	False match rate (FMR)	$\frac{FP}{FP+TN}$
	False non-match rate (FNMR)	$\frac{FN}{FN+TP}$
	False positive rate (FPR)	$\frac{FP}{FP+TN}$
	False rejection rate (FRR)	$\frac{FN}{FN+TP}$
	MCC (Matthews correlation coefficient)	$\frac{TP\times TN-FP\times FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$
	ROC (Receiver operating characteristic)	Curve of TPR vs. FPR
Multi classification evaluation	Micro-F1	$\text{Precision}_{micro}=\frac{\sum_i TP_i}{\sum_i (TP_i+FP_i)}$
		$\text{Recall}_{micro}=\frac{\sum_i TP_i}{\sum_i (TP_i+FN_i)}$
		$\frac{2\times\text{Precision}_{micro}\times\text{Recall}_{micro}}{\text{Precision}_{micro}+\text{Recall}_{micro}}$
	Macro-F1	$\text{Precision}_{macro}=\frac{1}{N}\sum_i \frac{TP_i}{TP_i+FP_i}$
		$\text{Recall}_{macro}=\frac{1}{N}\sum_i \frac{TP_i}{TP_i+FN_i}$
		$\frac{2\times\text{Precision}_{macro}\times\text{Recall}_{macro}}{\text{Precision}_{macro}+\text{Recall}_{macro}}$
	G-Mean (Geometric mean score)	$\sqrt{\text{Sensitivity}\times\text{Specificity}}$
Regression model evaluation	RMSE (Root mean squared error)	$\sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i-\hat{y}_i)^2}$
	MAE (Mean absolute error)	$\frac{1}{n}\sum_{i=1}^{n}\lvert y_i-\hat{y}_i\rvert$
	MAPE (Mean absolute percentage error)	$\frac{1}{n}\sum_{i=1}^{n}\frac{\lvert y_i-\hat{y}_i\rvert}{y_i}$

Author Contributions All authors contributed equally to this work.

Funding Open Access funding provided thanks to the CRUE-CSIC agreement with Springer Nature. This work was supported by the Spanish Ministry of Science and Innovation through the PID2021-125962OB-C31 "SECURING" and PID2023-147592OB-I00 "SE4GenAI" projects. Additional funding was provided by the ARTEMISA International Chair of Cybersecurity (C057/23) and the DANGER Strategic Project of Cybersecurity (C062/23), both funded by the Spanish National Institute of Cybersecurity through the European Union NextGenerationEU and the Recovery, Transformation, and Resilience Plan.

Declarations

Conflict of interest The authors have no others conflict of interest to declare that are relevant to the content of this article.

References

Abdulaal A, Liu Z, Lancewicki T (2021) Practical approach to asynchronous multivariate time series anomaly detection and localization. In: Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining. KDD '21, pp. 2485–2494. Association for Computing Machinery, New York, NY, USA https://doi.org/10.1145/3447548.3467174

Ahmed CM, Palleti VR, Mathur AP (2017) Wadi: a water distribution testbed for research in the design of secure cyber physical systems. In: Proceedings of the 3rd International Workshop on Cyber-Physical Systems for Smart Water Networks. CySWATER '17, pp. 25–28. Association for Computing Machinery, New York, NY, USA https://doi.org/10.1145/3055366.3055375

Akram J, Anaissi A, Othman W, Alabdulatif A, Akram A (2024) DroneSSL: self-supervised multimodal anomaly detection in internet of drone things. IEEE Trans Consum Electron 70(1):4287–4298. https://doi.org/10.1109/TCE.2024.3376440

Alrumaih TNI, Alenazi MJF (2024) CGAAD: centrality- and graph-aware deep-learning model for detecting cyberattacks targeting industrial control systems in critical infrastructure. IEEE Internet Things J 11(13):24162–24182. https://doi.org/10.1109/JIOT.2024.3390691

Altaf T, Wang X, Ni W, Yu G, Liu RP, Braun R (2024) GNN-based network traffic analysis for the detection of sequential attacks in IoT. Electronics (Switzerland). https://doi.org/10.3390/electronics13122274

Aslan HI, Choi C (2023) VisGIN visibility graph neural network on one-dimensional data for biometric authentication [Formula presented]. Exp Syst Appl. https://doi.org/10.1016/j.eswa.2023.121323

Barros CDT, Mendonça MRF, Vieira AB, Ziviani A (2021) A survey on embedding dynamic graphs. Preprint at https://arxiv.org/abs/2101.01229

Benzaïd C, Taleb T, Sami A, Hireche O (2023) FortisEDoS: a deep transfer learning-empowered economical denial of sustainability detection framework for cloud-native network slicing. IEEE Trans Dependable Secure Comput 21(4):2818–2835. https://doi.org/10.1109/TDSC.2023.3318606

Cao Y, Jiang H, Deng Y, Wu J, Zhou P, Luo W (2022) Detecting and mitigating DDoS attacks in SDN using spatial-temporal graph convolutional network. IEEE Trans Depend Secure Comput 19(6):3855–3872. https://doi.org/10.1109/TDSC.2021.3108782

Carletti V, Foggia P, Rosa F, Vento M (2024) Detecting malicious IoT network communication through graph neural networks in real-world conditions. Pattern Recogn Lett 189:92–98. https://doi.org/10.1016/j.patrec.2025.01.010

Chen L, Mao Y, Zhou H, Zhang B, Wang Z, Wu J (2023) MTS-GAT: multivariate time series anomaly detection based on graph attention networks. Int J Sens Netw 43(1):38–49. https://doi.org/10.1504/IJSNET.2023.133812

Chiranjeevi VR, Malathi D (2024) Anomaly graph: leveraging dynamic graph convolutional networks for enhanced video anomaly detection in surveillance and security applications. Neural Comput Appl 36(20):12011–12028. https://doi.org/10.1007/s00521-024-09738-3
Ding C, Sun S, Zhao J (2023) MST-GAT: A multimodal spatial-temporal graph attention network for time series anomaly detection. Inform Fusion 89:527–536. https://doi.org/10.1016/j.inffus.2022.08.011
Diukarev V, Starukhin Y (2024) Proposed methods for preventing overfitting in machine learning and deep learning. Asian J Res Comput Sci 17(10):85–94. https://doi.org/10.9734/ajrcos/2024/v17i10511
Fan Y, Fu T, Listopad NI, Liu P, Garg S, Hassan MM (2024) Utilizing correlation in space and time: Anomaly detection for Industrial Internet of Things (IIoT) via spatiotemporal gated graph attention network. Alex Eng J 106:560–570. https://doi.org/10.1016/j.aej.2024.08.048
Fan J, Wang Z, Wu H, Sun D, Wu J, Lu X (2023) An adversarial time-frequency reconstruction network for unsupervised anomaly detection. Neural Netw 168:44–56. https://doi.org/10.1016/j.neunet.2023.09.018
Fang Y, Zhao Z, Xu Y, Liu Z (2022) Log anomaly detection based on hierarchical graph neural network and label contrastive coding. Comput Mater Continua 74(2):4099–4118. https://doi.org/10.32604/cmc.2023.033124
Fard SH (2024) Machine learning on dynamic graphs: a survey on applications. Preprint at https://arxiv.org/abs/2401.08147
Feng Z, Wang R, Wang T, Song M, Wu S, He S (2024) A Comprehensive survey of dynamic graph neural networks: models, frameworks, benchmarks, experiments and challenges. https://arxiv.org/abs/2405.00476
Fu C, Li Q, Xu K (2024) Flow interaction graph analysis: unknown encrypted malicious traffic detection. IEEE/ACM Trans Netw 32(4):2972–2987. https://doi.org/10.1109/TNET.2024.3370851
Gama JA, Žliobaitundefined I, Bifet A, Pechenizkiy M, Bouchachia A (2014) A survey on concept drift adaptation. ACM Comput Surv. https://doi.org/10.1145/2523813
Gao R, He W, Yan L, Liu D, Yu Y, Ye Z (2023) Hybrid graph transformer networks for multivariate time series anomaly detection. J Supercomput 80(1):642–669. https://doi.org/10.1007/s11227-023-05503-w
Ge D, Cheng Y, Cao S, Ma Y, Wu Y (2024) An enhanced abnormal information expression spatiotemporal model for anomaly detection in multivariate time-series. Compl Intell Syst 10(2):2937–2950. https://doi.org/10.1007/s40747-023-01306-x
Guan S, He Z, Ma S, Gao M (2024) Multivariate time series anomaly detection with variational autoencoder and spatial-temporal graph network. Comput Secur. https://doi.org/10.1016/j.cose.2024.103877
Guan S, Zhao B, Dong Z, Gao M, He Z (2022) GTAD: graph and temporal neural network for multivariate time series anomaly detection. Entropy. https://doi.org/10.3390/e24060759
Guo W, Qiu H, Liu Z, Zhu J, Wang Q (2022) GLD-Net: Deep learning to detect DDoS attack via topological and traffic feature fusion. Comput Intell Neurosci 2022:4611331. https://doi.org/10.1155/2022/4611331
Guo J, Tang S, Li J, Pan K, Wu L (2024) Rustgraph: Robust anomaly detection in dynamic graphs by jointly learning structural-temporal dependency. IEEE Trans Knowl Data Eng 36(7):3472–3485. https://doi.org/10.1109/TKDE.2023.3328645
Guo H, Zhou Z, Zhao D, Gaaloul W (2024) EGNN: Energy-efficient anomaly detection for IoT multivariate time series data using graph neural network. Futur Gener Comput Syst 151:45–56. https://doi.org/10.1016/j.future.2023.09.028
Hassani K, Khasahmadi AH (2020) Contrastive multi-view representation learning on graphs. Preprint at https://arxiv.org/abs/2006.05582
He Y, Bian Y, Ding X, Wu B, Guan J, Zhang J, Zhou S (2024) Variate associated domain adaptation for unsupervised multivariate time series anomaly detection. ACM Trans Knowl Discov Data. https://doi.org/10.1145/3663573
He S, Li G, Wang J, Xie K, Sharma PK (2024) Uni-directional graph structure learning-based multivariate time series anomaly detection with dynamic prior knowledge. Int J Mach Learn Cybern. https://doi.org/10.1007/s13042-024-02212-5
He S, Li G, Yi T, Alfarraj O, Tolba A, Kumar Sangaiah A, Simon Sherratt R (2024) Graph structure learning-based multivariate time series anomaly detection in internet of things for human-centric consumer applications. IEEE Trans Consum Electron 70(3):5419–5431. https://doi.org/10.1109/TCE.2024.3409391
Huang X, Chen N, Deng Z, Huang S (2024) Multivariate time series anomaly detection via dynamic graph attention network and informer. Appl Intell 54(17):7636–7658. https://doi.org/10.1007/s10489-024-05575-y
Hundman K, Constantinou V, Laporte C, Colwell I, Soderstrom T (2018) Detecting spacecraft anomalies using LSTMS and nonparametric dynamic thresholding. In: Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. KDD '18, pp 387–395. Association for Computing Machinery, New York, NY, USA https://doi.org/10.1145/3219819.3219845
Jiang M, Jung T, Karl R, Zhao T (2022) Federated dynamic graph neural networks with secure aggregation for video-based distributed surveillance. ACM Trans Intell Syst Technol. https://doi.org/10.1145/3501808

Jiang W, Luo J (2022) Graph neural network for traffic forecasting: a survey. Expert Syst Appl 207:117921. https://doi.org/10.1016/j.eswa.2022.117921
Jo H, Lee S-W (2024) Edge conditional node update graph neural network for multivariate time series anomaly detection. Inf Sci. https://doi.org/10.1016/j.ins.2024.121062
Kalupahana Liyanage KS, Divakaran DM, Singh RP, Gurusamy M NSS Mirai Dataset. https://doi.org/10.21227/t970-nd64
Kang H, Ahn DH, Lee GM, Yoo JD, Park KH, Kim HK. IoT Network Intrusion Dataset. https://doi.org/10.21227/q70p-q449.
Khemani B, Patil S, Kotecha K, Tanwar S (2024) A review of graph neural networks: concepts, architectures, techniques, challenges, datasets, applications, and future directions. J Big Data 11:18
Khraisat A, Gondal I, Vamplew P, Kamruzzaman J (2019) Survey of intrusion detection systems: techniques, datasets and challenges. Cybersecurity. https://doi.org/10.1186/s42400-019-0038-7
King IJ, Huang HH (2023) Euler: detecting network lateral movement via scalable temporal link prediction. ACM Trans Priv Secur. https://doi.org/10.1145/3588771
Kipf T, Welling M (2016) Semi-supervised classification with graph convolutional networks. Preprint at https://arxiv.org/abs/1609.02907
Kipf TN, Welling M (2016) Variational graph auto-encoders. Preprint at https://arxiv.org/abs/1611.07308
Kong J, Wang K, Jiang M, Tao X (2024) GMAD: multivariate time series anomaly detection based on graph matching learning. Int J Mach Learn Cybern. https://doi.org/10.1007/s13042-024-02482-z
Koroniotis N, Moustafa N, Sitnikova E, Turnbull BP (2018) Towards the development of realistic botnet dataset in the internet of things for network forensic analytics: Bot-IoT dataset. Future Gener Comput Syst 100:779–796
Kunegis J (2013) Konect: the koblenz network collection. In: Proceedings of the 22nd International Conference on World Wide Web. WWW '13 Companion, pp 1343–1350. Association for Computing Machinery, New York, NY, USA https://doi.org/10.1145/2487788.2488173
Lanko V, Makarov I (2024) Graph attention diffusion for enhanced multivariate time series anomaly detection. IEEE Open J Ind Electron Soc. https://doi.org/10.1109/OJIES.2024.3501014
Li W, Hu W, Chen T, Chen N, Feng C (2022) StackVAE-G: An efficient and interpretable model for time series anomaly detection. AI Open 3:101–110. https://doi.org/10.1016/j.aiopen.2022.07.001
Li W, Zhang X-Y, Bao H, Shi H, Wang Q (2022) ProGraph: Robust network traffic identification with graph propagation. IEEE/ACM Trans Netw 31(3):1385–1399. https://doi.org/10.1109/TNET.2022.3216603
Li Y, Tarlow D, Brockschmidt M, Zemel R (2017) Gated graph sequence neural networks. Preprint at https://arxiv.org/abs/1511.05493
Liao J, Li J, Chen Y, Gu R, Zhu Y, Peng W (2024) DPDGAD: a dual-process dynamic graph-based anomaly detection for multivariate time series analysis in cyber-physical systems. Adv Eng Inform. https://doi.org/10.1016/j.aei.2024.102547
Liu J, Guo M (2024) DIGNN-A: Real-time network intrusion detection with integrated neural networks based on dynamic graph. Comput Mater Continua 82(1):817–842. https://doi.org/10.32604/cmc.2024.057660
Liu W, Liu X (2023) Intelligent detection method for malicious attacks in enterprise internet of things based on graph neural network. J Eng Sci Technol Rev 16(5):150–155. https://doi.org/10.25103/jestr.165.18
Liu Y, Pan S, Wang YG, Xiong F, Wang L, Chen Q, Lee VC (2021) Anomaly detection in dynamic graphs via transformer. IEEE Trans Knowl Data Eng 35(12):12081–12094. https://doi.org/10.1109/TKDE.2021.3124061
Liu F, Tian J, Miranda-Moreno L, Sun L (2023) Adversarial danger identification on temporally dynamic graphs. IEEE Trans Neural Netw Learn Syst 35(4):4744–4755. https://doi.org/10.1109/TNNLS.2023.3252175
Luo D, Cheng W, Xu D, Yu W, Zong B, Chen H, Zhang X (2020) Parameterized explainer for graph neural network. Preprint at https://arxiv.org/abs/2011.04573
Lyu S, Wang K, Wei Y, Liu H, Fan Q, Wang B (2023) GNN-based advanced feature integration for ICS anomaly detection. ACM Trans Intell Syst Technol. https://doi.org/10.1145/3620676
Mendoza M, Tesconi M, Cresci S (2020) Bots in social and interaction networks: detection and impact estimation. ACM Trans Inf Syst. https://doi.org/10.1145/3419369
Miao G, Wu G, Zhang Z, Tong Y, Lu B (2023) ADDAG-AE: Anomaly detection in dynamic attributed graph based on graph attention network and LSTM autoencoder. Electronics. https://doi.org/10.3390/electronics12132763
Mir AA, Zuhairi MF, Musa S, Alanazi MH, Namoun A (2024) Variational graph convolutional networks for dynamic graph representation learning. IEEE Access 12:161697–161717. https://doi.org/10.1109/ACCESS.2024.3483839
Mishra P, Varadharajan V, Tupakula U, Pilli ES (2019) A detailed investigation and analysis of using machine learning techniques for intrusion detection. IEEE Commun Surv Tutor 21(1):686–728. https://doi.org/10.1109/COMST.2018.2847722

MontazeriShatoori M, Davidson L, Kaur G, Lashkari AH (2020) Detection of doh tunnels using time-series classification of encrypted traffic. 2020 IEEE Intl Conf on Dependable, Autonomic and Secure Computing, Intl Conf on Pervasive Intelligence and Computing, Intl Conf on Cloud and Big Data Computing, Intl Conf on Cyber Science and Technology Congress (DASC/PiCom/CBDCom/CyberSciTech), pp 63–70

Moustafa N (2021) A new distributed architecture for evaluating AI-based security systems at the edge: Network ton_iot datasets. Sustain Cities Soc 72:102994. https://doi.org/10.1016/j.scs.2021.102994

Moustafa N, Slay J (2015) UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). In: 2015 Military Communications and Information Systems Conference (MilCIS), pp 1–6 https://doi.org/10.1109/MilCIS.2015.7348942

Pang J, Jia L, Deng J (2022) A survey on dynamic graph neural networks modeling. 2022 China Automation Congress (CAC). https://doi.org/10.1109/CAC57257.2022.10055755

Pang H, Wei S, Li Y, Liu T, Zhang H, Qin Y, Zhao Y (2024) Asymptotic consistent graph structure learning for multivariate time-series anomaly detection. IEEE Trans Instrum Meas 73:1–10. https://doi.org/10.1109/TIM.2024.3369159

Peng S, Nie J, Shu X, Ruan Z, Wang L, Sheng Y, Xuan Q (2021) A multi-view framework for BGP anomaly detection via graph attention network. Comput Netw 214:109129. https://doi.org/10.1016/j.comnet.2022.109129

Peng H, Zhang J, Huang X, Hao Z, Li A, Yu Z, Yu PS (2024) Unsupervised social bot detection via structural information theory. ACM Trans Inf Syst. https://doi.org/10.1145/3660522

Perotti A, Bajardi P, Bonchi F, Panisson A (2023) GRAPHSHAP: explaining identity-aware graph classifiers through the language of motifs. Preprint at https://arxiv.org/abs/2202.08815

Pinto A, Herrera L-C, Donoso Y, Gutierrez JA (2023) Survey on intrusion detection systems based on machine learning techniques for the protection of critical infrastructure. Sensors. https://doi.org/10.3390/s23052415

Presekal A, Ştefanov A, Rajkumar VS, Palensky P (2023) Attack graph model for cyber-physical power systems using hybrid deep learning. IEEE Trans Smart Grid 14(5):4007–4020. https://doi.org/10.1109/TSG.2023.3237011

Protogerou A, Kopsacheilis EV, Mpatziakas A, Papachristou K, Theodorou TI, Papadopoulos S, Drosou A, Tzovaras D (2022) Time series network data enabling distributed intelligence. A holistic IoT security platform solution. Electronics (Switzerland). https://doi.org/10.3390/electronics11040529

Qin S, Chen L, Luo Y, Tao G (2023) Multiview graph contrastive learning for multivariate time-series anomaly detection in IoT. IEEE Internet Things J 10(24):22401–22414. https://doi.org/10.1109/JIOT.2023.3303946

Reddy KKC, Kaza VS, Mohana MR, Alamer A, Alam S, Shuaib M, Basudan S, Sheneamer A (2024) Detecting and forecasting cryptojacking attack trends in Internet of Things and wireless sensor networks devices. PeerJ Computer Science 10:1–35. https://doi.org/10.7717/peerj-cs.2491

Sarhan M, Layeghy S, Portmann M (2022) Towards a standard feature set for network intrusion detection system datasets. Mob Netw Appl 27(1):357–370. https://doi.org/10.1007/s11036-021-01843-0

Scarselli F, Gori M, Tsoi AC, Hagenbuchner M, Monfardini G (2009) The graph neural network model. IEEE Trans Neural Netw 20(1):61–80. https://doi.org/10.1109/TNN.2008.2005605

Sharafaldin I, Lashkari AH, Ghorbani AA(2018) Toward generating a new intrusion detection dataset and intrusion traffic characterization. In: International Conference on Information Systems Security and Privacy . https://api.semanticscholar.org/CorpusID:4707749

Shi Y, Wang B, Yu Y, Tang X, Huang C, Dong J (2023) Robust anomaly detection for multivariate time series through temporal GCNs and attention-based VAE. Knowl Based Syst. https://doi.org/10.1016/j.knosys.2023.110725

Shin H-K, Lee W, Yun J-H, Kim H (2020) HAI 1.0: HIL-based augmented ICS security dataset. In: 13th USENIX Workshop on Cyber Security Experimentation and Test (CSET 20). USENIX Association, Boston, MA, USA https://www.usenix.org/conference/cset20/presentation/shin

Shlomi J, Battaglia P, Vlimant J-R (2021) Graph neural networks in particle physics. Mach Learn Sci Technol 2(2):021001. https://doi.org/10.1088/2632-2153/abbf9a

Song Y, Xin R, Chen P, Zhang R, Chen J, Zhao Z (2023) Identifying performance anomalies in fluctuating cloud environments: a robust correlative-GNN-based explainable approach. Futur Gener Comput Syst 145:77–86. https://doi.org/10.1016/j.future.2023.03.020

Stolfo S, Fan W, Lee W, Prodromidis A, Chan P (1999) KDD Cup 1999 Data. UCI machine learning repository. https://doi.org/10.24432/C51C7N

Su Y, Zhao Y, Niu C, Liu R, Sun W, Pei D (2019) Robust anomaly detection for multivariate time series through stochastic recurrent neural network. In: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. KDD '19, pp. 2828–2837. Association for Computing Machinery, New York, NY, USA https://doi.org/10.1145/3292500.3330672

Sun Y, Lin Z, Shi B, Zhang S, Ma S, Jin P, Zhong Z, Pan L, Guo Y, Pei D (2024) Interpretable failure localization for microservice systems based on graph autoencoder. ACM Trans Softw Eng Methodol. https://doi.org/10.1145/3695999

Tian Z, Zhuo M, Liu L, Chen J, Zhou S (2023) Anomaly detection using spatial and temporal information in multivariate time series. Sci Rep 13(1):4400. https://doi.org/10.1038/s41598-023-31193-8

Velickovic P, Cucurull G, Casanova A, Romero A, Lio' P, Bengio Y (2017) Graph attention networks. Preprint at https://arxiv.org/abs/1710.10903

Wang Y, Li Z, Barati Farimani A (2023) Graph neural networks for molecules. Springer, Cham, pp 21–66. https://doi.org/10.1007/978-3-031-37196-7_2

Wang Y, Li J, Zhao W, Han Z, Zhao H, Wang L, He X (2023) N-STGAT: Spatio-temporal graph neural network based network intrusion detection for near-earth remote sensing. Remote Sens. https://doi.org/10.3390/rs15143611

Wang J, Shao S, Bai Y, Deng J, Lin Y (2023) Multiscale wavelet graph AutoEncoder for multivariate time-series anomaly detection. IEEE Trans Instrum Meas 72:1–11. https://doi.org/10.1109/TIM.2022.3223142

Wang X, Wang X, He M, Zhang M, Lu Z (2023) Spatial-temporal graph model based on attention mechanism for anomalous IoT intrusion detection. IEEE Trans Industr Inf 20(3):3497–3509. https://doi.org/10.1109/TII.2023.3308784

Wang C, Xing S, Gao R, Yan L, Xiong N, Wang R (2023) Disentangled dynamic deviation transformer networks for multivariate time series anomaly detection. Sensors. https://doi.org/10.3390/s23031104

Wang M, Yang N, Weng N (2024) K-GetNID: Knowledge-guided graphs for early and transferable network intrusion detection. IEEE Trans Inf Forensics Secur. https://doi.org/10.1109/TIFS.2024.3431932

Wen M, Chen Z, Xiong Y, Zhang Y (2024) LGAT: A novel model for multivariate time series anomaly detection with improved anomaly transformer and learning graph structures. Neurocomputing. https://doi.org/10.1016/j.neucom.2024.129024

Wibowo RN, Sukarno P, Jadied EM (2019) NSL-KDD dataset. https://api.semanticscholar.org/CorpusID:198166203

Wu Z, Pan S, Chen F, Long G, Zhang C, Yu PS (2021) A comprehensive survey on graph neural networks. IEEE Trans Neural Netw Learn Syst 32(1):4–24. https://doi.org/10.1109/tnnls.2020.2978386

Wu W, Pang A, Yang W (2023) Heterogeneous sensor fault detection for networked systems based on a graph transformer. IEEE Sens J 24(4):5266–5278. https://doi.org/10.1109/JSEN.2023.3345377

Wu Z, Pan S, Long G, Jiang J, Chang X, Zhang C (2020) Connecting the dots: Multivariate time series forecasting with graph neural networks. In: Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. KDD '20, pp 753–763. Association for Computing Machinery, New York, NY, USA . https://doi.org/10.1145/3394486.3403118

Xie S, Li L, Zhu Y (2024) Anomaly detection for multivariate time series in IoT using discrete wavelet decomposition and dual graph attention networks. Comput Secur 146:104075. https://doi.org/10.1016/j.cose.2024.104075

Yan H, Li F, Chen J, Liu Z, Wang J, Feng Y, Zhang X (2023) A graph embedded in graph framework with dual-sequence input for efficient anomaly detection of complex equipment under insufficient samples. Reliab Eng Syst Safety 238:109418. https://doi.org/10.1016/j.ress.2023.109418

Yan B, Yang C, Shi C, Fang Y, Li Q, Ye Y, Du J (2023) Graph mining for cybersecurity: a survey. ACM Trans Knowl Discov Data 18(2):1–52. https://doi.org/10.1145/3610228

Yang J, Yue Z (2023) Learning hierarchical spatial-temporal graph representations for robust multivariate industrial anomaly detection. IEEE Trans Industr Inf 19(6):7624–7635. https://doi.org/10.1109/TII.2022.3216006

Yang Q, Zhang J, Zhang J, Sun C, Xie S, Liu S, Ji Y (2024) Graph transformer network incorporating sparse representation for multivariate time series anomaly detection. Electronics. https://doi.org/10.3390/electronics13112032

Ying R, Bourgeois D, You J, Zitnik M, Leskovec J (2019) GNNExplainer: Generating explanations for graph neural networks. Preprint at https://arxiv.org/abs/1903.03894

Yun S, Jeong M, Kim R, Kang J, Kim HJ (2019) Graph transformer networks. Preprint at https://arxiv.org/abs/1911.06455

Zhang B (2024) Securing RFID with GNN: a real-time tag cloning attack detection system. IEEE Open J Commun Soc. https://doi.org/10.1109/OJCOMS.2024.3502630

Zhang Z, Geng Z, Han Y (2024) Graph structure change-based anomaly detection in multivariate time series of industrial processes. IEEE Trans Industr Inf 20(4):6457–6466. https://doi.org/10.1109/TII.2023.3347000

Zhang W, He P, Qin C, Yang F, Liu Y (2023) A graph attention network-based model for anomaly detection in multivariate time series. Journal of Supercomputing 80(6):8529–8549. https://doi.org/10.1007/s11227-023-05772-5

Zhang D, Wang M, Bu Y, Yu J, Yang L (2024) PDGAT-ID: An intrusion detection method for industrial control systems based on periodic extraction and spatiotemporal graph attention. Comput Secur 149:104210. https://doi.org/10.1016/j.cose.2024.104210

Zhang W, Wang Y, Chen L, Yuan Y, Zeng X, Xu L, Zhao H (2023) Dynamic circular network-based federated dual-view learning for multivariate time series anomaly detection. Bus Inf Syst Eng 66(1):19–42. https://doi.org/10.1007/s12599-023-00825-8

Zhang G, Zhang S, Yuan G (2024) Bayesian graph local extrema convolution with long-tail strategy for misinformation detection. ACM Trans Knowl Discov Data. https://doi.org/10.1145/3639408

Zhang Z, Zhu Z, Xu C, Zhang J, Xu S (2024) Towards accurate anomaly detection for cloud system via graph-enhanced contrastive learning. Compl Intell Syst. https://doi.org/10.1007/s40747-024-01659-x

Zhang H, Shen T, Wu F, Yin M, Yang H, Wu C (2021) Federated graph learning—a position paper. Preprint at https://arxiv.org/abs/2105.11099

Zhao M, Fink O (2024) DyEdgeGAT: dynamic edge via graph attention for early fault detection in IIoT systems. IEEE Internet Things J 11(13):22950–22965. https://doi.org/10.1109/JIOT.2024.3381002

Zhao M, Peng H, Li L (2024) Multivariate time series anomaly detection based on dynamic graph neural networks and self-distillation in industrial internet of things. IEEE Internet Things J. https://doi.org/10.1109/JIOT.2024.3520362

Zhao M, Peng H, Li L, Ren Y (2024) Multivariate time series anomaly detection based on spatial-temporal network and transformer in industrial internet of things. Comput Mater Continua 80(2):2815–2837. https://doi.org/10.32604/cmc.2024.053765

Zhao M, Peng H, Li L, Ren Y (2024) Graph attention network and informer for multivariate time series anomaly detection. Sensors. https://doi.org/10.3390/s24051522

Zhao H, Wang Y, Duan J, Huang C, Cao D, Tong Y, Xu B, Bai J, Tong J, Zhang Q (2020) Multivariate time-series anomaly detection via graph attention network, pp 841–850. Preprint at https://doi.org/10.1109/ICDM50108.2020.00093

Zheng B, Ming L, Zeng K, Zhou M, Zhang X, Ye T, Yang B, Zhou X, Jensen CS (2024) Adversarial graph neural network for multivariate time series anomaly detection. IEEE Trans Knowl Data Eng 36(12):7612–7626. https://doi.org/10.1109/TKDE.2024.3419891

Zheng Y, Yi L, Wei Z (2024) A survey of dynamic graph neural networks. Preprint athttps://arxiv.org/abs/2404.18211

Zhong F, Liu Y, Liu L, Zhang G, Duan S (2022) DEDGCN: dual evolving dynamic graph convolutional network. Secur Commun Netw. https://doi.org/10.1155/2022/6945397

Zhou L, Zeng Q, Li B (2022) Hybrid Anomaly detection via multihead dynamic graph attention networks for multivariate time series. IEEE Access 10:40967–40978. https://doi.org/10.1109/ACCESS.2022.3167640

Zhu Y, Wang X, Li Q, Yao T, Liang S (2021) BotSpot++: A hierarchical deep ensemble model for bots install fraud detection in mobile advertising. ACM Trans Inf Syst. https://doi.org/10.1145/3476107

Zola F, Segurola-Gil L, Bruse JL, Galar M, Orduna-Urrutia R (2022) Network traffic analysis through node behaviour classification: a graph-based approach with temporal dissection and data-level preprocessing. Comput Secur 115:102632. https://doi.org/10.1016/j.cose.2022.102632